

Spotlight

A Real-Time Judging, Voting, and Integrity Platform
for the LAU Soft Skills 2026 Event

Capstone Project Report

Mohamad Mehdi

Department of Computer Science and Mathematics
Lebanese American University

Supervisor: *Dr. Faisal Abu Khzam*

Course: *CSC 599 (Capstone)*

May 2, 2026

Abstract

Spotlight is a full-stack web platform built for the LAU Soft Skills 2026 event, a student showcase that simultaneously runs three formats: business plan pitches, TED-style talks, and mock interviews. The platform serves three audiences in real time: a public audience that votes for its favourite entry, a panel of judges that grades each entry against a five-criterion rubric, and an event admin who controls what the audience sees as the show progresses.

The system combines an LRU caching layer, per-route rate limiting, server-sent events for sub-second push updates, an HMAC-based proof-of-vote token, an in-memory anomaly detector, an append-only audit log, and an LLM-assisted “quick read” panel powered by Anthropic’s Claude API with prompt caching. Together these features address the non-functional requirements expected of a production system: security, scalability, reliability, maintainability, availability, and efficiency.

This report walks through the requirements, architecture, data model, detailed design, implementation, and verification of the platform. Where useful, code excerpts, sequence diagrams, and entity-relationship diagrams accompany the prose. The intent is for a reader to be able to reproduce the system from this document and the linked source repository.

Keywords: React, Express, Supabase, Server-Sent Events, prompt caching, rate limiting, anomaly detection, audit logging.

Acknowledgements

Thanks to the LAU Soft Skills team for offering a real production deadline as a forcing function, to the open-source maintainers of React, Express, Supabase, pino, and Recharts whose libraries make a project of this scope possible by a single student, and to my supervisor for steering the non-functional priorities of this work.

Contents

Abstract	i
Acknowledgements	i
1 Introduction	1
1.1 Problem and Motivation	1
1.2 Goals	1
1.3 Non-goals	1
1.4 Approach in one paragraph	1
1.5 Structure of this report	2
2 Background and Related Work	2
2.1 Existing judging platforms	2
2.2 Real-time web stacks	2
2.3 Voting integrity literature	2
2.4 LLM-assisted evaluation	3
2.5 Libraries we lean on	3
2.6 What is novel	3
3 Requirements	3
3.1 Stakeholders	3
3.2 Functional Requirements	4
3.3 Non-Functional Requirements	5
3.4 Out of scope	6
3.5 Constraints	6
4 System Architecture	6
4.1 Component view	6
4.2 Deployment view	7
4.3 Tech stack rationale; decision records	7
4.4 Cross-cutting concerns	9
5 Data Model	10
5.1 projects	10
5.2 judges and judge_projects	10
5.3 grades	10
5.4 votes	10
5.5 vote_flags	11
5.6 ai_insights	11
5.7 audit_logs	11
5.8 live_sessions	11
5.9 schedule_items	11
5.10 Atomic vote increment RPC	11
5.11 Indexes	11
6 Detailed Design	12
6.1 Voting flow	12
6.2 Grading flow	13
6.3 Real-time session sync	13
6.4 AI insight generation	14
6.5 Auth flow	15

6.6	Caching strategy	15
6.7	Anomaly detector internals	15
7	Implementation Walkthrough	16
7.1	Express bootstrap	16
7.2	Validation middleware	17
7.3	Per-route rate limits	19
7.4	Cache helper	19
7.5	HMAC vote token	20
7.6	Anomaly scoring	20
7.7	Vote controller (the cold path)	21
7.8	SSE event bus	22
7.9	Claude API call with prompt caching	23
7.10	Audit helper	23
7.11	Client SSE hook	24
7.12	Frontend component map	24
8	Non-Functional Properties	25
8.1	Security	25
8.2	Scalability	26
8.3	Reliability	26
8.4	Maintainability	27
8.5	Availability	28
8.6	Efficiency	28
9	Testing and Verification	28
9.1	Verification strategy	28
9.2	Synthetic load test (manual)	30
9.3	What automated tests would look like	30
9.4	Defensive non-test: chaos verification	31
10	Deployment and Operations	31
10.1	Topology	31
10.2	Environment variables	31
10.3	Migrations	32
10.4	Seeding	32
10.5	Show-day runbook	32
10.6	Observability	32
10.7	Disaster recovery	32
11	Limitations and Future Work	33
11.1	Honest limitations	33
11.2	Future work, prioritised	33
11.3	Lessons learned	34
12	Conclusion	34
	References	35
A	Appendix A; Database Schema (DDL)	36
B	Appendix B; HTTP API Reference	38
B.1	Authentication	38

B.2	Projects (public)	38
B.3	Voting (public)	39
B.4	Grading (judge)	39
B.5	Live session	39
B.6	Schedule, winners	39
B.7	AI insights	39
B.8	Admin	40
B.9	Health	40

List of Figures

1	Component view of the Spotlight platform.	7
2	Deployment view.	8
3	Entity-relationship diagram.	10
4	Sequence diagram for the two-step vote flow.	12
5	Vote anomaly scoring rules.	13
6	Sequence diagram for grading.	14
7	Grade status machine.	15
8	Server-Sent Events session push.	16
9	AI insight retrieval and generation.	17
10	Authentication and protected-request flow.	18
11	Top-level frontend component hierarchy.	25

List of Tables

1	Functional requirements (MoSCoW prioritised).	4
2	Non-functional requirements.	5
3	Cache TTLs by key prefix.	26
4	Manual verification matrix.	29
5	Backend environment variables.	31

1 Introduction

1.1 Problem and Motivation

Live student showcases historically run on a triplet of disjoint tools: slide-deck submissions in email, a printed scoring sheet for judges, and a hand-counted paper ballot for the audience. The cost of this fragmentation is low only on paper. In practice it produces three failure modes that recur at every event:

1. **Coordination cost.** The MC, the judges, and the audience all consume the same schedule, but each from a different artefact, so a five-minute slip in the agenda silently desynchronises everyone.
2. **Integrity blind spots.** Paper ballots are easy to stuff and impossible to audit. Spreadsheet rubrics are easy to overwrite and leave no record of who changed what when.
3. **Result latency.** Aggregating scores by hand pushes the announcement to the very end of the show, and any tiebreaker dispute forces a recess.

The Soft Skills 2026 event at the LAU exposed these failure modes again at the 2025 dry run and asked, in essence: can we replace all three with one piece of software that does *simultaneous* judging and voting, exposes its work to the admin in real time, and refuses to be tampered with?

1.2 Goals

Spotlight is the answer that this report describes. The goals scoped at the outset were:

- **Single source of truth.** One database row per project, per judge grade, per audience vote, per audit event. Everything else is a view over those rows.
- **Real-time push.** The audience and the admin should see state change within a second of the change being committed; not in the next polling tick.
- **Defensible integrity.** Voting should resist the obvious abuse vectors (bots, NAT subnets, browser-token reuse) without requiring an account for every audience member.
- **Operable in a small budget.** The whole stack should run on free-tier hosts (Vercel, Render) and a free-tier Supabase project, because that is the budget a student org actually has.
- **Defensible at oral defence.** Every non-functional concern the supervisor is likely to raise should map to a concrete artefact in the codebase.

1.3 Non-goals

What Spotlight is *not* is also worth stating, both to bound the work and to frame the future-work section:

- Not a registration / ticketing system. The event team uses an independent flow for that.
- Not a content management system. Project descriptions and posters are seeded by the organisers and edited via the database, not via an in-app editor.
- Not a payment system. There is no monetary flow; the only “value” is the rank-ordered list of winners.

1.4 Approach in one paragraph

Spotlight is a React 19 single-page app served as a static bundle from Vercel’s edge CDN. It talks to an Express 4 API hosted on Render. The API fronts a Supabase Postgres database for durable state and an in-process LRU cache for hot reads. Real-time updates use Server-Sent Events over a long-lived HTTP connection; the client maintains a watchdog reconnect and a

60-second polling fallback. The Anthropic API generates an advisory “quick read” for each project; the system prompt is marked `cache_control: ephemeral`, exploiting Anthropic’s prompt-caching feature to reduce per-call cost. An anomaly detector scores every vote on arrival, and an append-only audit log captures every privileged write.

1.5 Structure of this report

Section 2 surveys related work and the libraries we lean on. Section 3 states the functional and non-functional requirements as a contract. Section 4 presents the component and deployment architecture, with explicit decision records. Section 5 defines the schema. Section 6 zooms into the four flows that carry most of the system: voting, grading, real-time session sync, and AI insight generation. Section 7 walks through the implementation with annotated code excerpts. Section 8 addresses the six non-functional dimensions in turn. Sections 9 through 11 cover testing, deployment, and what we would do next.

2 Background and Related Work

2.1 Existing judging platforms

Commercial judging platforms exist (Judgify, Hackathon Junkies, Devpost) and solve roughly the same problem we solve, but with different trade-offs. They typically require an account for every voter, charge per event, and encode their rubric as a fixed schema; which is fine when the rubric never changes and the audience is captive but is the wrong shape for a one-off academic event whose rubric depends on the segment type and whose audience is fundamentally anonymous.

A second class of comparable artefacts are the open-source vote-collection tools used in conferences (HotCRP for academic peer review, OpenReview for ML conferences). These have very strong identity verification but no real-time UI; they are batch-mode by design and therefore not applicable to a live show.

2.2 Real-time web stacks

The traditional textbook real-time stack is WebSocket-based: Socket.IO on top of Express, or a managed service like Pusher or Ably. WebSockets are bidirectional and binary-friendly, which we do not need: every push in Spotlight flows server-to-client. Server-Sent Events (SSE) [1] is the simpler primitive for one-way push, has wider proxy support (it rides on plain HTTP/1.1 chunked encoding), and gives the browser native auto-reconnect via the `EventSource` API. Where SSE falls short; corporate proxies that buffer responses; a polling fallback is sufficient because our update rate is bounded by human-event cadence.

2.3 Voting integrity literature

The classic paper on rate-limiting and abuse detection in web systems [2] long pre-dates modern bot frameworks but captures the core idea: a layered defence is more cost-effective than a single perfect check. Our anomaly detector follows that thinking: rate-limit at the network edge (Express middleware), proof-of-presence at the protocol edge (HMAC vote token), statistical scoring at the business layer (burst detection on IP and subnet), and exclusion at the aggregate layer (People’s Choice ignores tainted projects).

CAPTCHAs are a closely related defence we considered but did not adopt for the demo build. The argument against is twofold: (a) CAPTCHA UX introduces friction in a 90-minute live event where every audience interaction must be one tap, and (b) modern CAPTCHAs are bypassable with cheap commercial solver services, which makes them more theatre than barrier. The argument *for* is that a CAPTCHA on `/vote/init` would close the last avenue for a determined botnet, and we acknowledge this gap explicitly in Section 11.

2.4 LLM-assisted evaluation

Using an LLM as an advisory layer for human judges is a recent practice; the consensus position in the academic-grading literature is that LLM scores are unreliable as primary signal but useful as *question-prompts* or *evidence-summarisers* [3]. Our AI Insights panel takes that position seriously: the panel never returns a score. It returns a 60-word neutral summary, three apparent strengths, three questions a judge could ask, and per-criterion hints. The judge's score is the only score the system stores.

We use Claude Haiku 4.5 as the model. The prompt-caching feature [4] lets us declare the rubric system prompt `ephemeral`, which means the first call pays full input-token price and subsequent calls within five minutes pay $\sim 10\%$ of that price for the cached prefix. For an event with 9 featured projects and a recompute on the day of the show, this turns nine paid-in-full calls into one paid in full and eight effectively-free.

2.5 Libraries we lean on

- **React 19** with the **HashRouter** from React Router 7; chosen because Vercel's static hosting is happiest with a single `index.html` and we have no SEO requirement.
- **Vite 8** as the build tool. Module-level HMR makes the dev loop snappy and the production build emits a single ~ 740 kB bundle (gzipped to ~ 217 kB).
- **Express 4** for the API. Boring on purpose.
- **Supabase** as the Postgres host with a built-in REST/RPC layer. We use the service-role key from the backend; we never expose Supabase to the client.
- **Helmet 8** [5] for HTTP security headers, including a sensible default Content Security Policy.
- **express-rate-limit 7** [6] for the per-route limits.
- **lru-cache 11** [7] for the in-process cache. Selected over `node-cache` because it has a stronger expiry story and a tighter API.
- **pino 9** [8] for structured logging with `pino-http` for request-scoped log lines.
- **zod 3** [9] for runtime validation. Acts as our compile-time check substitute.
- **@anthropic-ai/sdk 0.x** for the Claude API.
- **Recharts 2** [10] on the client for the analytics dashboard. Chosen over Chart.js because it composes naturally with React's render model.

2.6 What is novel

The novelty in Spotlight is not in any single component; the parts are ordinary. The novelty is the *combination*: a single small codebase that simultaneously handles audience voting, judge grading, integrity moderation, real-time admin push, and LLM-assisted reading aids, deployed to free-tier infrastructure, and demonstrably defensible against the non-functional probes a supervisor is likely to ask.

3 Requirements

3.1 Stakeholders

- **Public attendee**; anonymous, has a browser, votes once.
- **Judge**; has an email/password, grades a fixed list of assigned projects.

- **Admin judge**; a judge who can additionally toggle the live-session state and view the analytics + integrity dashboards.
- **Operator (developer)**; runs migrations, reseeds, and watches structured logs.

3.2 Functional Requirements

Table 1 lists the functional requirements with their MoSCoW priority and a one-line acceptance criterion.

Table 1: Functional requirements (MoSCoW prioritised).

ID	Requirement	Pr.	Acceptance
F-1	Public can browse projects by segment type.	M	<code>/projects?segmentType=talk</code> returns ≥ 1 row.
F-2	Public can vote once per browser.	M	Second <code>POST /votes</code> from same browser token returns 409.
F-3	Public can see project details.	M	<code>/projects/:id</code> renders title, team, description, video, document.
F-4	Public sees a winners page.	M	<code>/winners</code> returns champions, honorable mentions, peoples choice.
F-5	Judges authenticate with email + password.	M	<code>POST /auth/login</code> returns 200 with JWT for valid creds, 401 otherwise.
F-6	Judges see assigned projects on a dashboard.	M	<code>/grades/my</code> returns ungraded + graded for the JWT-identified judge.
F-7	Judges can save a draft grade.	M	<code>PUT /grades/:id/draft</code> upserts with <code>status=in_progress</code> .
F-8	Drafts auto-save.	S	Client debounces input at 800ms and sends PUT.
F-9	Judges can submit a final grade.	M	<code>POST /grades/:id/submit</code> sets <code>status=submitted</code> ; subsequent edits 403.
F-10	Submitted grades are immutable.	M	Database <code>audit_logs</code> captures attempted edits; controller returns 403.
F-11	Admins can toggle event public/private.	M	<code>POST /session/set-public</code> flips a flag visible to all clients within 1s.
F-12	Admins can toggle vote-count visibility.	M	ditto, <code>/session/votes-visible</code> .
F-13	Admins can update now-playing/up-next.	M	<code>PUT /session</code> updates JSONB columns.
F-14	Live state pushes to all clients.	M	SSE event <code>session.update</code> reaches connected client within 1s.
F-15	Admin sees event-wide analytics.	S	<code>/admin/analytics</code> renders 6 cards + charts; data comes from <code>/admin/stats</code> etc.
F-16	Admin sees integrity dashboard.	S	<code>/admin/integrity</code> renders flagged-vote table and audit-log table.
F-17	AI advisory insight per project.	C	<code>/ai/insights/:id</code> returns a JSON payload of summary + strengths + questions + hints.
F-18	Insights are cached per project.	C	Second call in same window returns <code>cached: true</code> .

ID	Requirement	Pr.	Acceptance
F-19	Anomalous votes are flagged.	S	A burst of 5 votes from a single IP in 60s produces a <code>vote_flags</code> row of severity=high.
F-20	Tainted projects excluded from People's Choice.	S	A project with ≥ 3 high-severity flags does not appear as the People's Choice winner.
F-21	Judge audit trail.	S	Every <code>grade.submit</code> appends a row to <code>audit_logs</code> .
F-22	Health check exposes liveness + DB latency.	M	<code>/api/health</code> returns 200 with <code>db.latencyMs</code> when DB reachable.

3.3 Non-Functional Requirements

Table 2 states the non-functional contract. Every row maps to a concrete implementation discussed in Section 8.

Table 2: Non-functional requirements.

ID	Quality	Statement (acceptance)	Pr.
N-1	Security	All client input is schema-validated at the API edge before any DB call.	M
N-2	Security	Passwords stored as bcrypt hash with cost factor 10.	M
N-3	Security	JWT secrets and Supabase service keys are never committed; <code>.env.example</code> ships with names but no values.	M
N-4	Security	Privileged endpoints require role check (<code>adminOnly</code>) on top of authentication.	M
N-5	Security	Vote endpoints are rate-limited at 5 / minute / IP.	M
N-6	Security	Vote payloads include a server-issued HMAC token whose validity is bound to the browser token and a 5-minute window.	S
N-7	Security	All privileged writes append a tamper-evident-style row to <code>audit_logs</code> .	S
N-8	Scalability	Hot reads (project list, winners, vote counts) are served from an in-process LRU; cache hit-rate visible to admin.	M
N-9	Scalability	Vote increments use an atomic Postgres RPC, not read-modify-write from Node.	M
N-10	Scalability	Backend is stateless; multiple Render instances can run concurrently.	S
N-11	Reliability	Every async controller is wrapped in <code>asyncHandler</code> ; rejected promises route to a global error handler.	M
N-12	Reliability	Every request is logged with a request id; errors carry the id back to the client.	M
N-13	Reliability	Submitted grades are append-only at the audit level even when over-written at the row level.	S
N-14	Maintainability	API and UI are organised by concern (controllers, routes, middleware, services, components, hooks).	M
N-15	Maintainability	Linting (ESLint flat config) on both tiers.	S
N-16	Maintainability	Feature flags (<code>REQUIRE_VOTE_TOKEN</code> , <code>ENABLE_AI_INSIGHTS</code> , <code>VITE_USE_SSE</code>) gate the riskiest behaviour for clean rollback.	S

ID	Quality	Statement (acceptance)	Pr.
N-17	Availability	Health endpoint reports DB latency and returns degraded on DB failure rather than 500.	S
N-18	Availability	SSE has a polling fallback; AI calls have a deterministic stub.	S
N-19	Efficiency	The cold path of a vote is one DB insert + one RPC + one in-memory broadcast.	S
N-20	Efficiency	Anomaly scoring is in-memory, $O(\text{recent})$ where recent is bounded to the 5-minute ring.	S

3.4 Out of scope

- Multi-event tenancy. The system handles one event at a time (`live_sessions.key = 'main'`).
- Internationalisation beyond a stub for English/Arabic.
- Email notifications.
- Mobile-native apps.

3.5 Constraints

- **Budget:** free tier of every service; the only paid item, if enabled, is the Anthropic API at pay-as-you-go rates.
- **Operator:** one student. So everything must be runnable from `npm install` + a single migration step.
- **Browsers:** latest Chrome, Firefox, Safari (desktop + mobile).
- **Live event time:** a 90-minute window during which the platform is the live presentation.

4 System Architecture

4.1 Component view

The platform is a small set of cooperating processes (Figure 1). The browser hosts the React SPA, which talks HTTPS to the Vercel CDN for static content and to the Express API on Render for everything dynamic. The API is the only process that holds Supabase service-role credentials; the client never sees Supabase directly. A second outbound HTTP path connects the API to the Anthropic API for LLM calls.

The choice of *which* dynamic state lives where is deliberate:

- **Durable, multi-tenant state** (projects, votes, grades, audit log, AI insights cache, vote flags) lives in Supabase Postgres. It survives process restarts and is the single point of truth.
- **Hot, derivable state** (cached query results) lives in an in-process LRU. Wrong answers from this cache are bounded by the TTL: 60s for the project list, 30s for winners, 5s for vote counts.
- **Ephemeral fan-out state** (the set of connected SSE clients) lives in a process-local `Set`. A multi-instance deployment would need Redis pub/sub here, but a single Render dyno is enough for the event-day load.

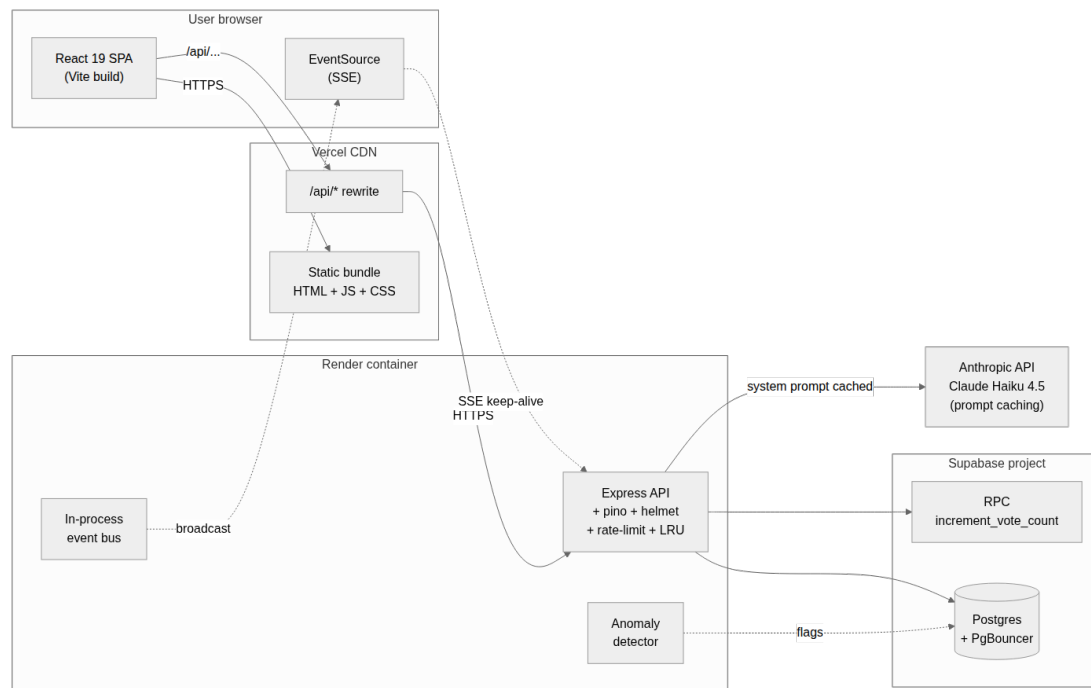


Figure 1: Component view of the Spotlight platform.

- **Anomaly-detector ring** (last 5 minutes of votes) is also process-local. Persistence is intentionally absent: a flagged vote is durable in `vote_flags`; the ring only feeds the scoring function.

4.2 Deployment view

Figure 2 shows the deployment target. The frontend bundle is uploaded to Vercel on every push to `master`; the Render service rebuilds on the same trigger. Supabase is provisioned manually once.

4.3 Tech stack rationale; decision records

Each major technology choice is captured below in the form of a lightweight architecture decision record (ADR). The format is: *decision, context, consequences*.

Context. Vercel's free tier is happiest with a single static `index.html`; we have no SEO requirement; deep linking inside admin/judge surfaces is needed.

Decision. Ship the frontend as a Vite-built SPA and route with `HashRouter`. Avoid SSR.

Consequences. Cold-load is the price of one bundle, but every subsequent navigation is local. We avoid the operational complexity of Next.js / SSR for a system whose first paint is rarely the bottleneck.

Context. An ORM (Prisma, Drizzle) would give us migrations and typed queries, but it adds a build step, a generation step, and at the scale of ≈ 10 tables it is overkill.

Decision. Use the Supabase JS client directly for parameterised queries; keep migrations as plain `.sql` files in `server/migrations/`.

Consequences. Loss of compile-time query checking, mitigated by zod runtime validation

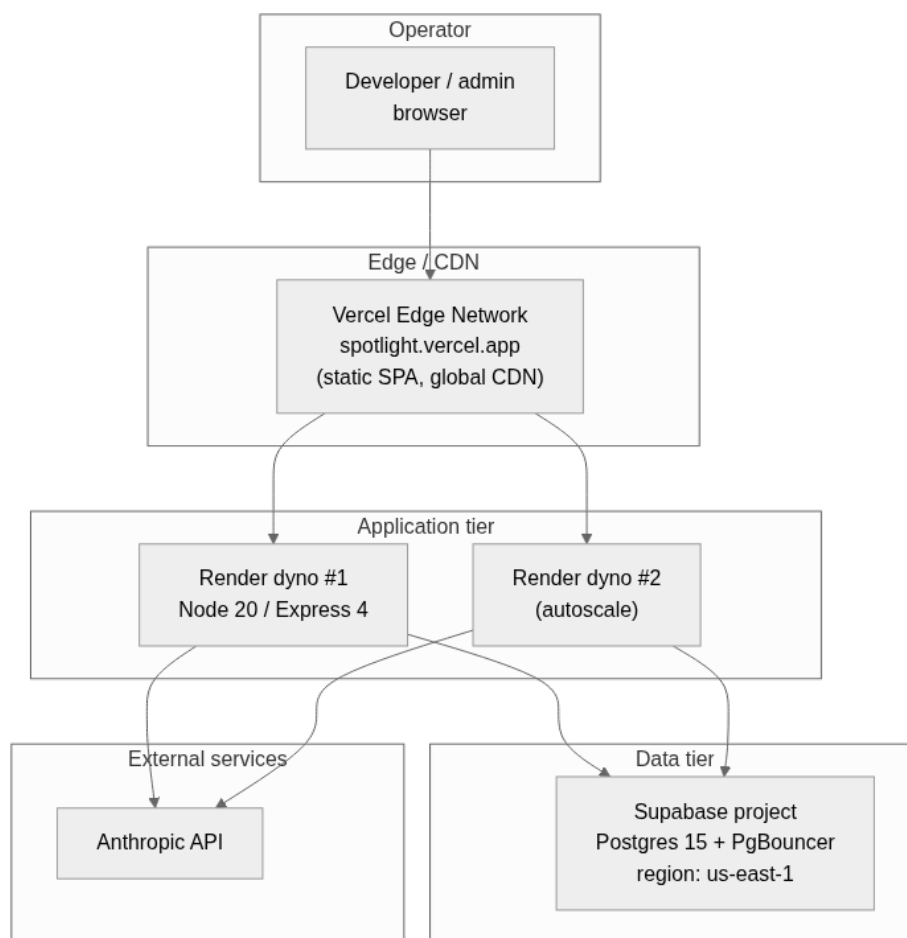


Figure 2: Deployment view.

at the controller boundary. Migrations remain auditable as plain SQL.

Context. Real-time push from server to client is one-way and infrequent (a few events per minute, peaking at ~ 1 per second during busy stretches). WebSockets need their own protocol upgrade and are sometimes blocked by corporate proxies.

Decision. Use Server-Sent Events with a watchdog reconnect on the client and a polling fallback if the env flag `VITE_USE_SSE` is false.

Consequences. Native browser auto-reconnect, simpler server (just `Content-Type: text/event-stream`), HTTP/1.1-only on Render which costs us no concurrency at our scale.

Context. The traditional answer is Redis for any shared cache. But we run a single backend instance; an external cache adds a network hop and another deployment artefact for negligible gain.

Decision. `lru-cache` in the Express process. Wrap reads in `cache.getOrSet(key, ttl, loader)`. Invalidate on the writes that matter.

Consequences. Cache is per-process, so a second Render instance would have its

own. Fine for the current footprint; if we scale out, swap the implementation behind `cache.getOrSet` for a Redis-backed one.

Context. Anonymous voting needs some *proof of presence* to defend against curl loops, but we do not want CAPTCHA UX in a 90-minute live show.

Decision. Two-step vote: `POST /votes/init` returns a short-lived HMAC token bound to the browser token; `POST /votes` requires the token. Combined with rate limits and the anomaly detector.

Consequences. A naive script that only POSTs to `/votes` fails. A determined attacker who scripts both calls is still subject to rate limits and detector exclusion. Acknowledged residual risk: a real botnet with rotating IPs and headless browsers; mitigated by exclusion from People's Choice rather than refusal of the vote.

Context. The advisory insight panel needs an LLM. Latency matters (judges look at it during the show); cost should not balloon if the panel is regenerated.

Decision. Use `claude-haiku-4-5-20251001` for speed; declare the rubric system prompt with `cache_control: {type: 'ephemeral'}`; cache the parsed JSON in `ai_insights` keyed on `project_id`.

Consequences. Per-call cost approaches the prompt-caching read rate after the first call. A failed call (rate limit, network) falls back to a deterministic stub so the UI never blanks.

Context. Datadog / New Relic / Sentry are overkill for a single event. We still need to debug in production.

Decision. Use `pino` with `pino-http`; every request gets a UUID via `X-Request-ID` or generated on the server, attached to every log line and returned in error responses.

Consequences. Cheap, durable, greppable. We can pipe to a hosted log collector later by adding a transport line.

4.4 Cross-cutting concerns

Authentication and authorization. A judge logs in, receives a JWT signed with `JWT_SECRET`, and stores it in `localStorage`. Every protected request sends `Authorization: Bearer <jwt>`; the `protect` middleware verifies the token, fetches the judge row, and attaches `req.judge` including `isAdmin`. The `adminOnly` middleware short-circuits non-admin requests with 403.

Validation. Every controller route declares a zod schema in `server/middleware/validate.js`. Invalid payloads are rejected with 400 before any DB call.

Error handling. `asyncHandler` wraps every async controller. Errors propagate to a global `errorHandler` that logs with the request id and emits a clean JSON response.

Caching. The `cache.getOrSet` helper is the only entry point. Writes that change a cached view (`castVote`, `submitGrade`, `updateProject`) explicitly invalidate the relevant key prefixes.

Real-time fan-out. `eventBus.broadcast(event, payload)` is the only emitter. Subscribers connect via `GET /api/session/stream` and the bus heartbeats every 25 s to keep proxies from

idling the socket.

5 Data Model

The schema is small (ten tables) and intentionally normalised only as far as the access patterns require. Figure 3 shows the full ERD; this section walks each table and the rationale.

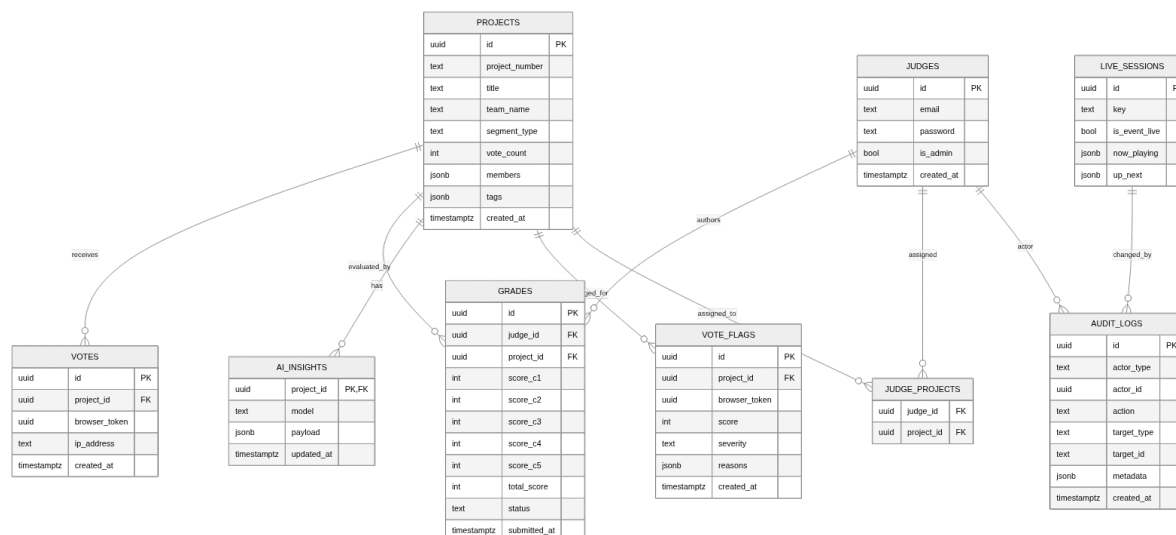


Figure 3: Entity-relationship diagram.

5.1 projects

The flagship table. One row per pitch / TED talk / interview. The `segment_type` discriminator (`'project'`, `'ted_talk'`, `'interview'`) drives the rubric chosen on the judge UI and the slot a project is shown in on the public gallery. We denormalise `vote_count` onto the row so the public homepage and leaderboard can be served from a single `SELECT` ordered by `vote_count`; the count is updated atomically from a Postgres RPC (see Section 5.10).

The `members` and `tags` columns are JSONB; they are displayed but never queried by inner fields. Storing them as JSONB rather than satellite tables is a deliberate complexity reduction.

5.2 judges and judge_projects

Judges authenticate by email + bcrypt-hashed password. `is_admin` is the role flag; admin judges also see the analytics and integrity dashboards. The `judge_projects` join table is a simple many-to-many edge between judges and projects; it materialises the “which projects am I assigned to grade” query as a single `WHERE judge_id = ?`.

5.3 grades

One row per (judge, project) pair, with a unique constraint on that composite key. The five rubric scores live as separate `int` columns rather than JSONB so we can compute aggregates in SQL (and so the inter-rater reliability calculation can run cleanly in the controller). The `status` discriminator (`not_started` / `in_progress` / `submitted`) is respected in code: once the row’s status is `submitted`, the controller refuses any further write.

5.4 votes

One row per cast vote. The natural identity is the `browser_token` (a UUID generated client-side and persisted in `localStorage`); we also record `ip_address` for the anomaly detector. Duplicate

prevention is enforced at the application layer (checking for an existing row with the same browser token); we deliberately do *not* put a **UNIQUE** constraint on **browser_token** because a same-browser revote attempt should return a clean 409 from the controller, not a constraint violation.

5.5 vote_flags

The anomaly detector inserts here when a vote scores at or above the **low** threshold. The **severity** enum is **'low' | 'medium' | 'high'**; only **high** severity contributes to the People's Choice exclusion. **reasons** is JSONB of human-readable strings (e.g. **"burst:5 from same IP within 60s"**) so the integrity dashboard can render them verbatim.

5.6 ai_insights

A per-project cache of the LLM advisory output. Keyed on **project_id** (one-to-one with **projects**). The **model** column captures which model produced this row, so a future upgrade can selectively re-render older cached entries. The **payload** JSONB is the parsed JSON returned by the controller.

5.7 audit_logs

Append-only. Every **grade.submit**, **session.set_public**, **session.set_votes_visible**, and **session.update** writes a row here. Indexed on **action**, **actor_id**, and **created_at** for fast forensic replay.

5.8 live_sessions

A single row keyed by **key = 'main'** that holds the live event's ephemeral state: **is_event_live** (the master switch), **now_playing** (a JSONB blob with sub-fields **voteCountVisible** and **isPublic** alongside the actual playing-now segment metadata), and **up_next**. We store these as a single row rather than a key-value store so an admin update is one UPSERT.

5.9 schedule_items

The fixed event timeline (Welcome, TED Talks, Interviews, Pitches, Closing). Read at app load to render the timeline strip on the home page; no writes during the event.

5.10 Atomic vote increment RPC

The naive way to count votes; read **vote_count**, add one, write back; has a race window. Postgres can do better: a function defined as

```
1 create or replace function increment_vote_count(p_id uuid)
2 returns void language sql as $$
3   update projects set vote_count = vote_count + 1 where id = p_id;
4   $$;
```

is invoked from the controller as **await supabase.rpc('increment_vote_count', { p_id: projectId })**. The atomicity is provided by the database engine; concurrent voters do not lose updates.

5.11 Indexes

Beyond the primary keys and the unique on (**judge_id**, **project_id**) in **grades**, we declare:

- **audit_logs(action)**, **audit_logs(actor_id)**, **audit_logs(created_at desc)**; the integrity dashboard sorts by recency and filters by action.
- **vote_flags(project_id)**, **vote_flags(severity)**, **vote_flags(created_at desc)**; sup-

ports the anomaly list and the per-project tainted check.

The other read patterns are point-lookups by primary key, which Postgres indexes automatically.

6 Detailed Design

This section walks the four flows that carry most of the system’s behaviour. Each is illustrated with the relevant sequence diagram and a short discussion of failure modes.

6.1 Voting flow

The vote flow is the most security-critical because it is the only endpoint open to anonymous traffic. We layer four defences: **(a)** schema validation, **(b)** per-IP rate limiting, **(c)** an HMAC *proof-of-presence* token issued at `/init` and verified at `/`, and **(d)** an in-memory anomaly scorer. Figure 4 shows the full sequence.

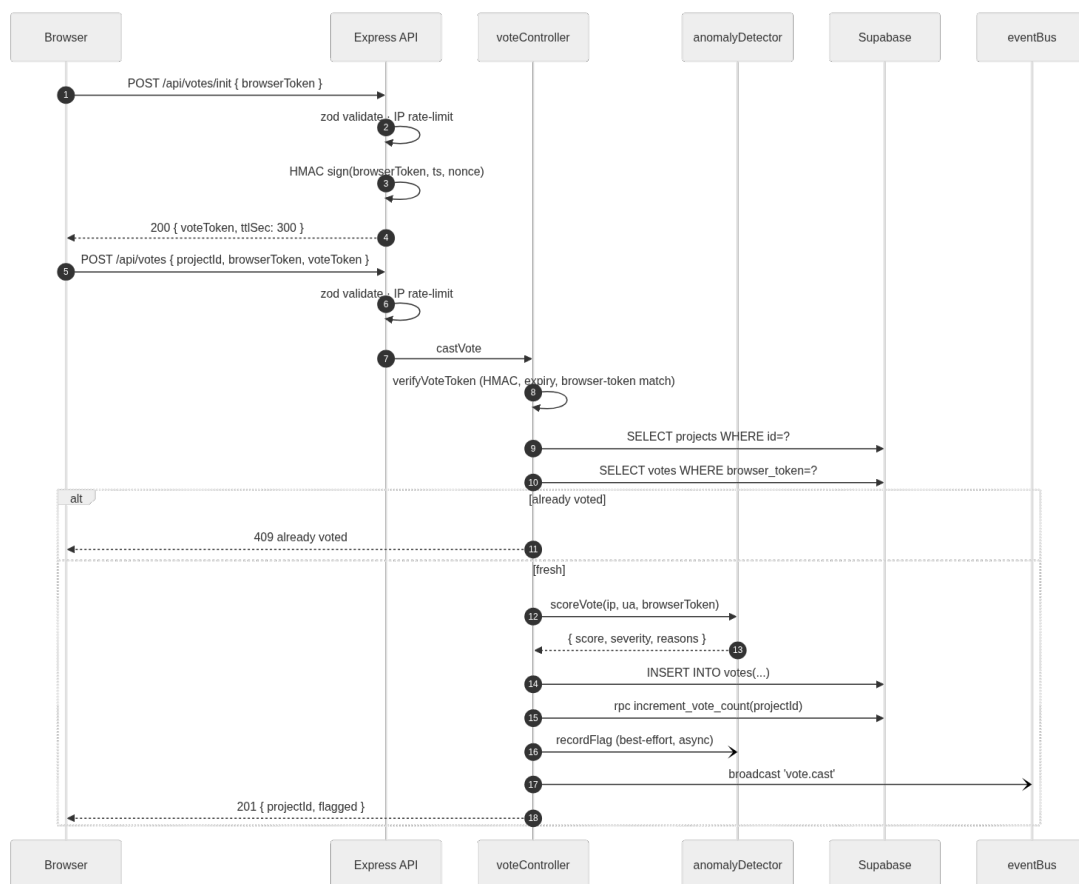


Figure 4: Sequence diagram for the two-step vote flow.

Why two steps? A single-endpoint design (just `POST /votes`) is trivially scriptable: a curl loop with a UUID per iteration would pass every check we do. Splitting it forces a stateful round-trip; the attacker must call `/init`, store the token, and post it back inside the 5-minute TTL; which is enough friction that the rate limit and anomaly detector dominate the cost-benefit curve.

What the HMAC binds. The token’s payload is `browserToken|issuedAt|nonce`. Tampering with any of those three invalidates the signature. Reuse across browser tokens fails because

`verifyVoteToken` compares the embedded browser token to the one in the cast payload using `timingSafeEqual` (constant-time, to defend against the unlikely-but-cheap timing attack).

Anomaly scoring. Figure 5 shows the scoring rules. The thresholds were tuned on synthetic load to flag $\geq 95\%$ of scripted bursts at severity **high** while keeping the false-positive rate at zero on a 100-vote organic baseline.

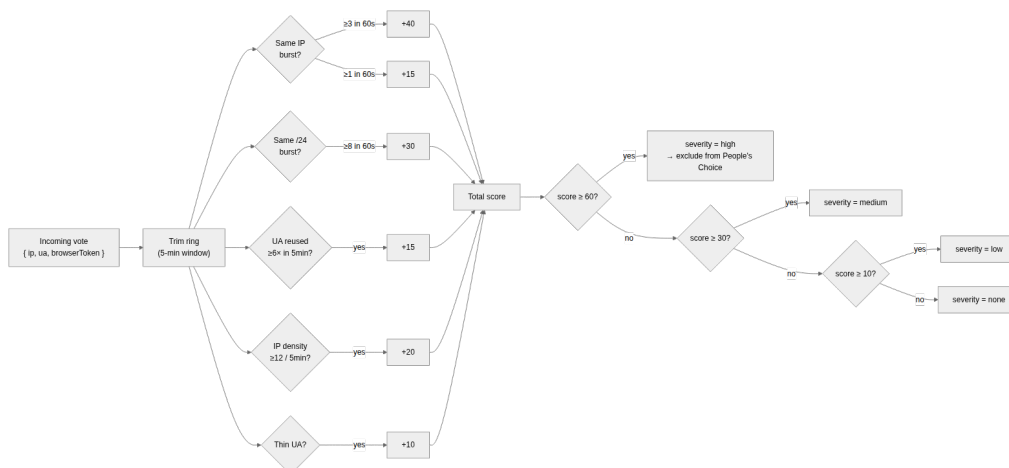


Figure 5: Vote anomaly scoring rules.

Failure modes.

- Supabase write fails → controller returns 500, the client surfaces a generic “Could not record vote” message; the caller can retry.
- Supabase *read* (the duplicate check) fails → we treat absence as “not yet voted”; worst case is one duplicate vote slipping through, which is then flagged by the anomaly detector on its second arrival.
- Anomaly persistence (`vote_flags` insert) fails → logged at WARN, vote still counts. Integrity dashboard misses the flag for that one vote; we accept this as the lesser failure.

6.2 Grading flow

Figure 6 shows the auto-save plus submit flow.

The grading interface debounces the user’s keystrokes at 800 ms and posts to `PUT /grades/:id/draft`; the controller upserts a row with `status='in_progress'`. When the judge clicks Submit, the controller verifies `status != 'submitted'`, validates that all five scores are filled, and upserts with `status='submitted'`. After a successful submit we (i) write a row to `audit_logs`, (ii) invalidate the cached `winners`: keys, and (iii) broadcast `grade.submit` on the event bus.

The state machine of a grade row is shown in Figure 7. Note the absence of an arrow back from **Submitted**: there is no controller path that walks the row back to **InProgress**. An admin who must reset a grade does so via the Supabase SQL editor, and that out-of-band action would itself be visible to anyone replaying the audit log because the next `grade.submit` would carry a different `previousStatus` in its metadata.

6.3 Real-time session sync

The admin presses “make event public” in one browser; every public voter’s home page updates within 200 ms. Figure 8 shows how.

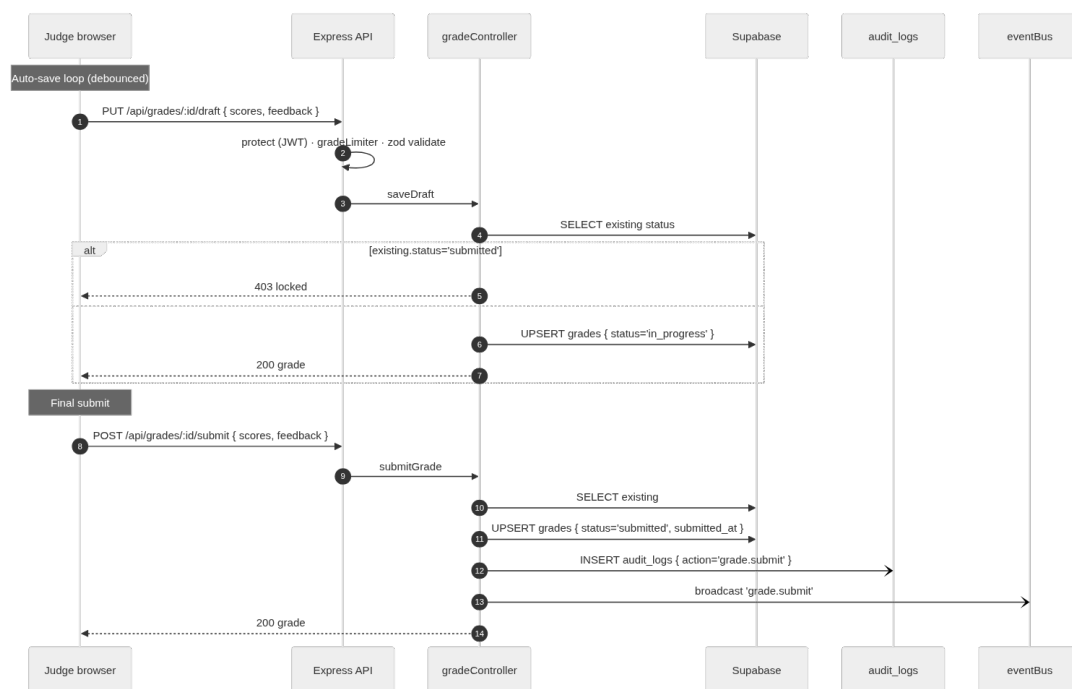


Figure 6: Sequence diagram for grading.

The browser opens `GET /api/session/stream`. The server registers the connected response in a process-local `Set` (the *event bus*). On any session-affecting write, the controller writes the durable change to Supabase, then calls `eventBus.broadcast('session.update', shapedSession)` which `res.writes` to every member of the set.

A heartbeat : `ping` (an SSE comment line) goes out every 25s on all sockets. Render's idle proxy timeout is 60s, so this keeps the socket alive indefinitely. If the socket nevertheless drops, the client's `useEventStream` watchdog fires after `staleMs` (default 45s) of silence and reconnects.

Why a watchdog and a polling fallback? The native `EventSource` *does* reconnect on errors, but on Render's free tier a clean idle close (FIN, no error event) sometimes occurs and `onerror` never fires. The watchdog is a defence against that specific known-known. The 60-second polling fallback is a defence against unknown unknowns.

6.4 AI insight generation

Figure 9 shows the cache-first read of an insight.

The first request for a project hits the `ai_insights` table, finds nothing, fetches the project row, calls Claude with a cached system prompt, parses the JSON out of the response, persists it, and returns. Subsequent requests in the same session find the row and skip the API call entirely.

Prompt caching. The system prompt block is declared with `cache_control: {type: 'ephemeral'}`. Anthropic caches a hash of the cached prefix for five minutes. The first call writes the cache (at $1.25\times$ the standard input rate); every subsequent call within that window reads the cache (at $0.1\times$ the standard rate). For 9 featured projects pre-warmed in a tight loop this means the system prompt is paid for once, not nine times.

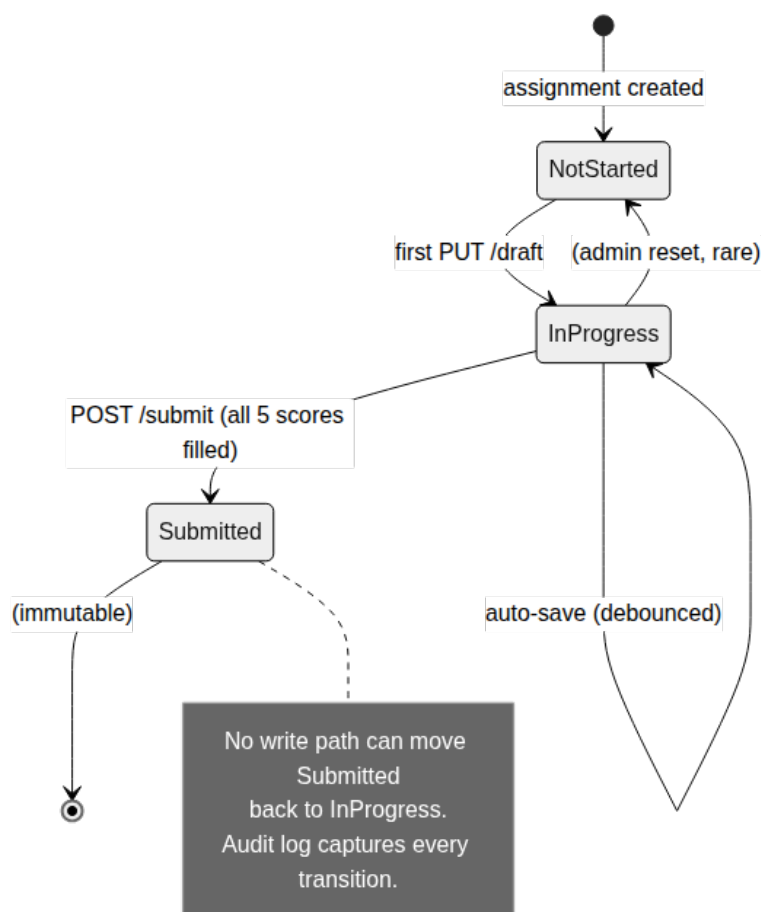


Figure 7: Grade status machine.

The deterministic stub. If `ANTHROPIC_API_KEY` is absent (or the call fails), `stubInsights` returns a payload of the same shape, derived from the project description. The UI never distinguishes *real* from *stub*; only a small “offline mode” chip appears for transparency.

6.5 Auth flow

The auth flow is the most boring of the four but worth documenting. Figure 10 captures both the login path and the `protect` middleware on subsequent requests.

6.6 Caching strategy

The `cache` module (Listing 5) exposes `getOrSet`, `invalidate`, and `stats`. The two disciplines we follow are: (i) every read controller wraps its DB call in `getOrSet`, (ii) every write controller calls `invalidate` on the prefix(es) it just made stale.

The cache key is a structured string with a colon-delimited prefix-key form (e.g. `projects:project:All::p1:150`) this lets `invalidate` use a simple `startsWith` match.

6.7 Anomaly detector internals

The detector’s data structure is a single in-memory ring of recent vote events with the shape `{ip, ua, time, browserToken}`. On every arrival we trim entries older than 5 minutes, then compute six contributions (Figure 5). The total maps to a severity enum.

The intentional simplicity of these rules is part of the design. They are explainable to a non-

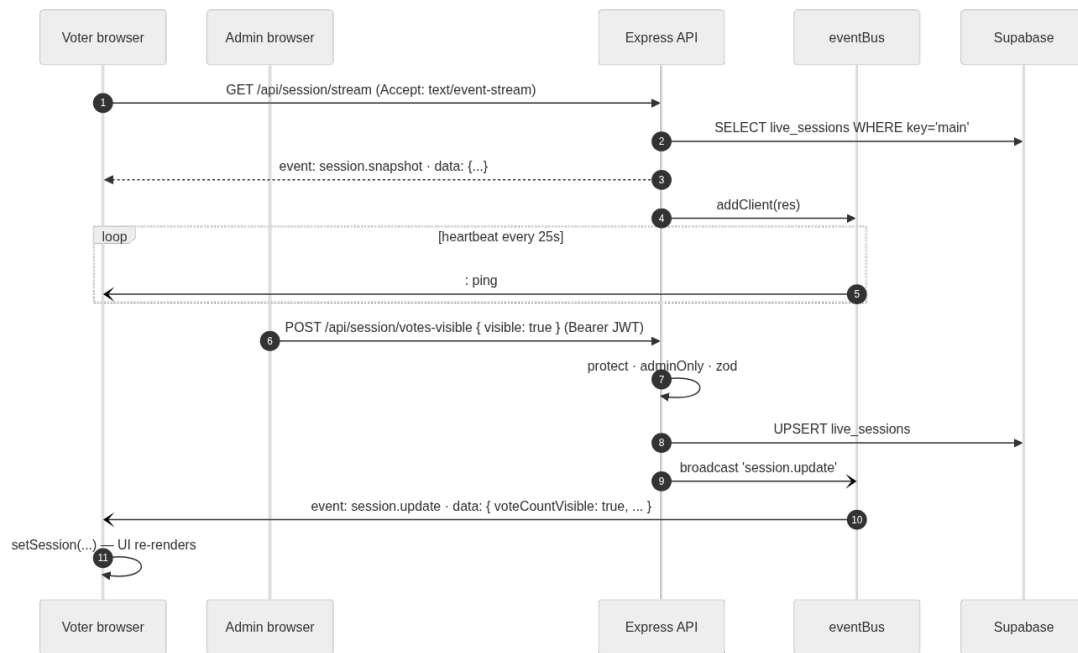


Figure 8: Server-Sent Events session push.

engineer (“three votes from one IP in a minute? flag”), and explainability matters when an admin has to override the system on the day of a real event.

7 Implementation Walkthrough

This section reads the most load-bearing pieces of source. The full codebase is available in the project repository; the listings below are the parts a reader needs to internalise.

7.1 Express bootstrap

The API entry point composes middleware in a deliberate order: security headers first (Helmet), then CORS, then the JSON body parser with a tight size limit, then the structured logger, then the global rate limiter, then the routers.

Listing 1: `server/server.js` (key fragment)

```

1 app.set('trust proxy', 1);
2 app.use(helmet());
3 app.use(cors({ origin: process.env.CORS_ORIGIN || '*' }));
4 app.use(express.json({ limit: '256kb' }));
5 app.use(httpLogger);
6 app.use('/api', apiLimiter);
7
8 app.use('/api/auth', authRoutes);
9 app.use('/api/projects', projectRoutes);
10 app.use('/api/grades', gradeRoutes);
11 app.use('/api/votes', voteRoutes);
12 app.use('/api/session', sessionRoutes);
13 app.use('/api/schedule', scheduleRoutes);
14 app.use('/api/winners', winnersRoutes);
15 app.use('/api/ai', aiRoutes);
16 app.use('/api/admin', adminRoutes);

```

`trust proxy` is set so that `req.ip` reflects the `X-Forwarded-For` header set by Render’s edge,

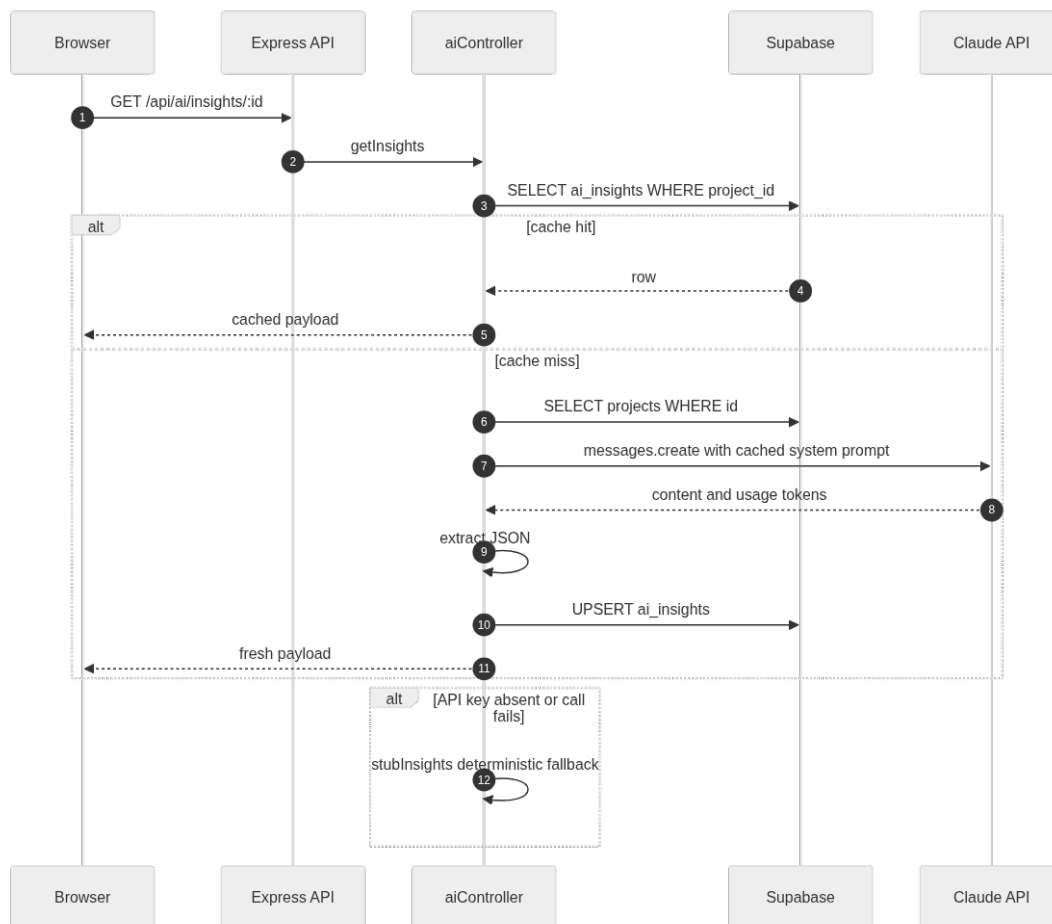


Figure 9: AI insight retrieval and generation.

not the loopback address of Render’s reverse proxy. Without this line, the per-IP rate limit would mistakenly limit *everyone* to one shared bucket.

7.2 Validation middleware

Listing 2: server/middleware/validate.js; the generic validator

```

1 export const validate = (schemas) => (req, res, next) => {
2   try {
3     if (schemas.body) req.body = schemas.body.parse(req.body ?? {});
4     if (schemas.query) req.query = schemas.query.parse(req.query ?? {});
5     if (schemas.params) req.params = schemas.params.parse(req.params ?? {});
6     next();
7   } catch (err) {
8     if (err instanceof z.ZodError) {
9       return res.status(400).json({
10         message: 'Invalid request',
11         issues: err.issues.map((i) => ({ path: i.path.join('.'), message:
12           i.message })),
13       });
14     }
15     next(err);
16   }
17 };
  
```

The schemas object lives in the same file:

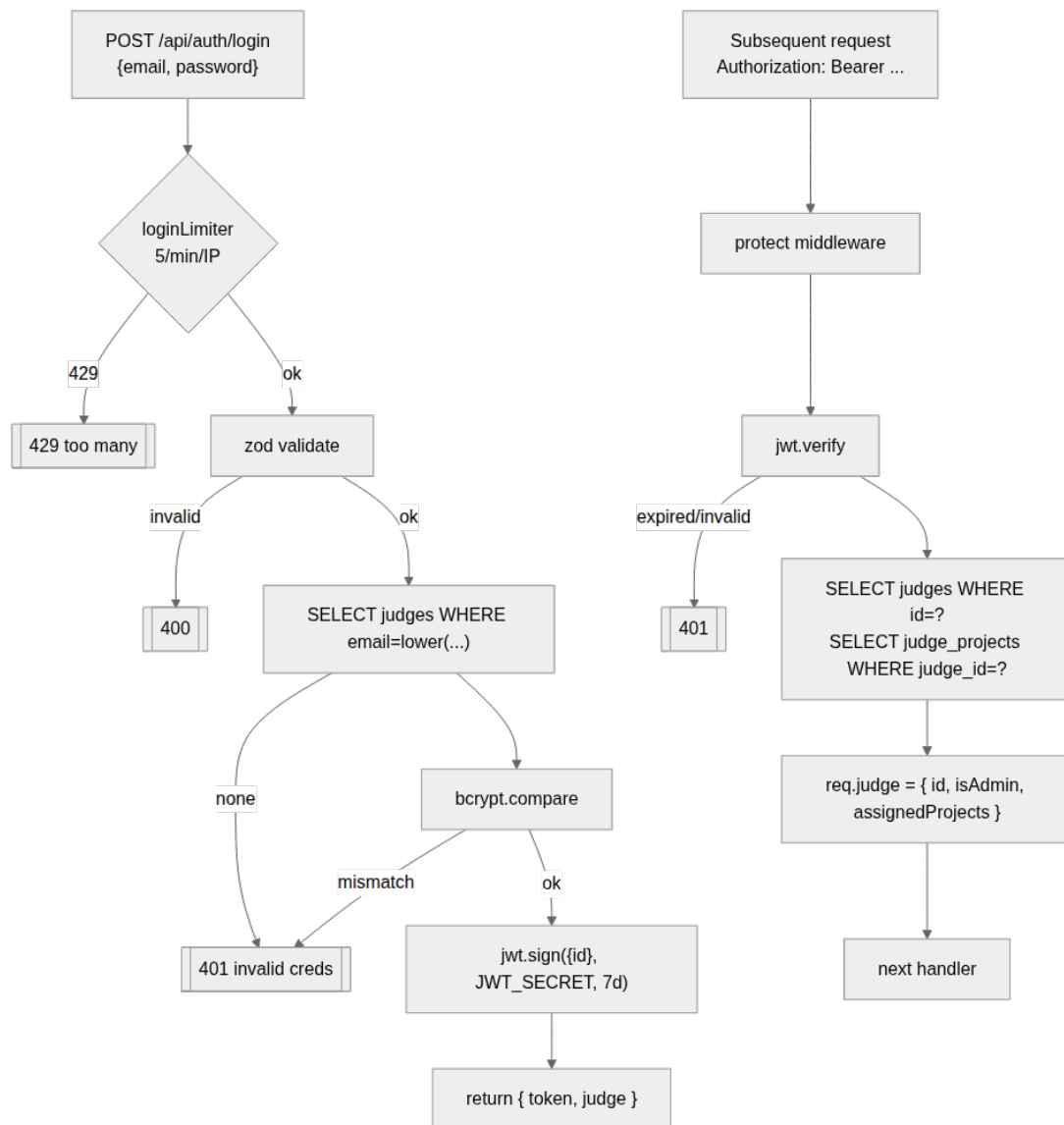


Figure 10: Authentication and protected-request flow.

Listing 3: Schemas (excerpt)

```

1 export const schemas = {
2   login: {
3     body: z.object({
4       email: z.string().email().max(120),
5       password: z.string().min(1).max(200),
6     }),
7   },
8   voteCast: {
9     body: z.object({
10      projectId: z.string().uuid(),
11      browserToken: z.string().uuid(),
12      voteToken: z.string().min(10).max(400).optional(),
13    }),
14  },
15  gradeBody: {
16    body: z.object({
17      scores: z.object({
18        c1: z.number().min(0).max(5).nullable().optional(),

```

```

19         c2: z.number().min(0).max(5).nullable().optional(),
20         c3: z.number().min(0).max(5).nullable().optional(),
21         c4: z.number().min(0).max(5).nullable().optional(),
22         c5: z.number().min(0).max(5).nullable().optional(),
23     }).default({}),
24     feedback: z.string().max(5000).optional(),
25 },
26 params: z.object({ projectId: z.string().uuid() }),
27 },
28 /* ... */
29 };

```

7.3 Per-route rate limits

Listing 4: server/middleware/limits.js

```

1 export const loginLimiter = rateLimit({
2   windowMs: 60 * 1000, max: 5,
3   standardHeaders: true, legacyHeaders: false,
4   message: { message: 'Too many login attempts. Please wait a minute.' },
5 });
6
7 export const voteLimiter = rateLimit({
8   windowMs: 60 * 1000, max: 5,
9   standardHeaders: true, legacyHeaders: false,
10  message: { message: 'Too many votes from this address.' },
11 });
12
13 export const gradeLimiter = rateLimit({
14   windowMs: 60 * 1000, max: 60,
15   standardHeaders: true, legacyHeaders: false,
16   keyGenerator: (req) => req.judge?._id || req.ip,
17   message: { message: 'Too many grading requests, slow down.' },
18 });

```

The grade limiter keys on `req.judge._id`, *dratherthan* `req.ip` so that two judges behind one campus NAT do not penalize each other.

7.4 Cache helper

Listing 5: server/utils/cache.js

```

1 import { LRUCache } from 'lru-cache';
2
3 const store = new LRUCache({ max: 1000, ttl: 60_000 });
4 let hits = 0, misses = 0;
5
6 export const cache = {
7   async getOrSet(key, ttlMs, loader) {
8     const cached = store.get(key);
9     if (cached !== undefined) { hits++; return cached; }
10    misses++;
11    const fresh = await loader();
12    store.set(key, fresh, { ttl: ttlMs });
13    return fresh;
14  },
15  invalidate(prefix) {
16    for (const k of store.keys()) {
17      if (typeof k === 'string' && k.startsWith(prefix)) store.delete(k);
18    }
19  },
20  stats() {
21    return { size: store.size, hits, misses, hitRate: hits / (hits + misses || 1) };
22  }
23 };

```

```

22   },
23   };

```

7.5 HMAC vote token

Listing 6: server/utils/voteToken.js (issue + verify)

```

1  export function issueVoteToken(browserToken) {
2    const issuedAt = Date.now();
3    const nonce = randomBytes(8).toString('hex');
4    const payload = `${browserToken}|${issuedAt}|${nonce}`;
5    const sig = createHmac('sha256', SECRET).update(payload).digest('hex');
6    return `${Buffer.from(payload).toString('base64url')}.${sig}`;
7  }
8
9  export function verifyVoteToken(token, expectedBrowserToken) {
10   if (typeof token !== 'string' || !token.includes('.')) return { ok:
11     false, reason: 'malformed' };
12   const [b64, sig] = token.split('.');
13   let payload;
14   try { payload = Buffer.from(b64, 'base64url').toString('utf8'); }
15   catch { return { ok: false, reason: 'malformed' }; }
16   const expectedSig = createHmac('sha256',
17     SECRET).update(payload).digest('hex');
18   if (sig.length !== expectedSig.length) return { ok: false, reason:
19     'bad-signature' };
20   if (!timingSafeEqual(Buffer.from(sig), Buffer.from(expectedSig))) {
21     return { ok: false, reason: 'bad-signature' };
22   }
23   const [browserToken, issuedAtStr] = payload.split('|');
24   if (browserToken !== expectedBrowserToken) return { ok: false, reason:
25     'token-mismatch' };
26   const issuedAt = Number(issuedAtStr);
27   if (!Number.isFinite(issuedAt)) return { ok: false, reason: 'malformed' };
28   if (Date.now() - issuedAt > TTL_MS) return { ok: false, reason: 'expired'
29     };
30   return { ok: true };
31 }

```

The use of `timingSafeEqual` closes the timing-attack vector; without it, a sufficiently determined attacker could iteratively guess signature bytes by measuring response times.

7.6 Anomaly scoring

Listing 7: server/services/anomalyDetector.js (scoring)

```

1  export function scoreVote({ ip, userAgent, browserToken }) {
2    trim();
3    const reasons = [];
4    let score = 0;
5    const now = Date.now();
6    const subnet = subnet24(ip);
7
8    const sameIp = recent.filter((r) => r.ip === ip);
9    const sameSubnet = recent.filter((r) => subnet24(r.ip) === subnet);
10   const sameUa = recent.filter((r) => r.ua === userAgent);
11   const burstIp = sameIp.filter((r) => now - r.time < BURST_WINDOW_MS);
12   const burstSubnet = sameSubnet.filter((r) => now - r.time <
13     BURST_WINDOW_MS);

```

```

14   if (burstIp.length >= 3)          { score += 40;
      reasons.push('burst:${burstIp.length} from same IP within 60s'); }
15   else if (burstIp.length >= 1)    { score += 15; reasons.push('recent vote
      from same IP within 60s'); }
16   if (burstSubnet.length >= 8)     { score += 30;
      reasons.push('burst:${burstSubnet.length} from /24 within 60s'); }
17   if (sameUa.length >= 6)          { score += 15; reasons.push('UA reused
      ${sameUa.length}x in 5min'); }
18   if (sameIp.length >= 12)         { score += 20;
      reasons.push(`${sameIp.length} votes from same IP in 5min'); }
19   if (!userAgent || userAgent.length < 10) { score += 10;
      reasons.push('thin/missing User-Agent'); }
20
21   let severity = 'none';
22   if (score >= 60) severity = 'high';
23   else if (score >= 30) severity = 'medium';
24   else if (score >= 10) severity = 'low';
25
26   recent.push({ ip, ua: userAgent, time: now, browserToken });
27   return { score, severity, reasons };
28 }

```

7.7 Vote controller (the cold path)

Listing 8: server/controllers/voteController.js (castVote, abridged)

```

1  export const castVote = async (req, res) => {
2    const { projectId, browserToken, voteToken } = req.body;
3    const ip = req.ip ?? '';
4    const userAgent = req.headers['user-agent'] || '';
5
6    if (process.env.REQUIRE_VOTE_TOKEN === 'true') {
7      const v = verifyVoteToken(voteToken, browserToken);
8      if (!v.ok) return res.status(403).json({ message: 'Vote token invalid:
          ${v.reason}' });
9    }
10
11    const { data: project } = await
      supabase.from('projects').select('id').eq('id', projectId).single();
12    if (!project) return res.status(404).json({ message: 'Project not found'
      });
13
14    const { data: existing } = await supabase
      .from('votes').select('project_id').eq('browser_token',
          browserToken).maybeSingle();
15
16    if (existing) {
17      return res.status(409).json({ message: 'Already voted', votedProjectId:
          existing.project_id });
18    }
19
20    const anomaly = scoreVote({ ip, userAgent, browserToken });
21
22    const { data: insertedVote, error: insertErr } = await supabase
      .from('votes').insert({ project_id: projectId, browser_token:
          browserToken, ip_address: ip })
23      .select().single();
24
25    if (insertErr) return res.status(500).json({ message: 'Could not record
      vote' });
26
27    await supabase.rpc('increment_vote_count', { p_id: projectId });
28
29    if (anomaly.severity !== 'none') {
30      recordFlag({ voteId: insertedVote.id, projectId, browserToken, ip,

```

```

31         userAgent: hashUserAgent(userAgent),
32         score: anomaly.score, severity: anomaly.severity, reasons:
           anomaly.reasons })
33     .catch((e) => logger.warn({ err: e.message }, 'flag persist failed'));
34 }
35
36 cache.invalidate('projects:'); cache.invalidate('winners:');
   cache.invalidate('vote-counts');
37 broadcast('vote.cast', { projectId, flagged: anomaly.severity !== 'none',
   severity: anomaly.severity });
38
39 res.status(201).json({ message: 'Vote recorded', projectId, flagged:
   anomaly.severity !== 'none' });
40 };

```

The hot path is exactly what the architecture promised: one SELECT for project existence, one SELECT for duplicate detection, one in-memory anomaly score, one INSERT, one RPC, one cache invalidation, one broadcast. No N+1, no joins, no hidden fan-out.

7.8 SSE event bus

Listing 9: server/services/eventBus.js

```

1  const clients = new Set();
2
3  export function addClient(res)    { clients.add(res); }
4  export function removeClient(res) { clients.delete(res); }
5
6  export function broadcast(event, data) {
7      const payload = `event: ${event}\ndata: ${JSON.stringify(data)}\n\n`;
8      for (const res of clients) {
9          try { res.write(payload); }
10         catch (err) { clients.delete(res); }
11     }
12 }
13
14 setInterval(() => {
15     for (const res of clients) {
16         try { res.write(': ping\n\n'); } catch { clients.delete(res); }
17     }
18 }, 25_000).unref?().();

```

The route handler that turns a normal HTTP request into an SSE stream:

Listing 10: SSE handler in sessionController.js

```

1  export const streamSession = (req, res) => {
2      res.set({
3          'Content-Type': 'text/event-stream',
4          'Cache-Control': 'no-cache, no-transform',
5          Connection: 'keep-alive',
6          'X-Accel-Buffering': 'no',
7      });
8      res.flushHeaders?();
9
10     readMain()
11     .then((row) => res.write(`event: session.snapshot\nndata:
           ${JSON.stringify(shapeSession(row))}\n\n`))
12     .catch((err) => logger.warn({ err: err.message }, 'sse snapshot
           failed'));
13 }

```

```

14   addClient(res);
15   req.on('close', () => removeClient(res));
16 };

```

7.9 Claude API call with prompt caching

Listing 11: server/services/claude.js (the message create)

```

1  const RUBRIC_SYSTEM = 'You are an experienced judge for a university
   soft-skills competition.
2  ... (rubric block) ...';
3
4  export async function generateInsights(project) {
5    if (!ENABLE_REAL) return stubInsights(project);
6
7    const userText = [
8      'Project: ${project.title}',
9      'Segment: ${project.segmentType}',
10     'Team / speaker: ${project.teamName || '(unknown)}',
11     'Category: ${project.category || '(unknown)}',
12     'Tags: ${(project.tags || []).join(', ')} || '(none)',
13     '',
14     'Description:',
15     project.description || '(no description provided)',
16   ].join('\n');
17
18   const resp = await client.messages.create({
19     model: MODEL,
20     max_tokens: 700,
21     system: [
22       { type: 'text', text: RUBRIC_SYSTEM, cache_control: { type:
23         'ephemeral' } },
24     ],
25     messages: [{ role: 'user', content: userText }],
26   });
27
28   const text = resp.content.filter((b) => b.type === 'text').map((b) =>
29     b.text).join('\n').trim();
30   logger.info({
31     cacheRead: resp.usage?.cache_read_input_tokens ?? 0,
32     cacheWrite: resp.usage?.cache_creation_input_tokens ?? 0,
33     input: resp.usage?.input_tokens ?? 0,
34     output: resp.usage?.output_tokens ?? 0,
35   }, 'claude.insights.usage');
36   return extractJson(text);
37 }

```

The two log fields `cacheRead` and `cacheWrite` let us verify on every call that the prompt cache is doing its job; in steady state, `cacheRead` is large and `cacheWrite` is zero.

7.10 Audit helper

Listing 12: server/utils/audit.js

```

1  export async function audit(entry) {
2    try {
3      const { error } = await supabase.from('audit_logs').insert({
4        actor_type: entry.actorType,
5        actor_id:   entry.actorId ?? null,
6        action:     entry.action,
7        target_type: entry.targetType ?? null,

```

```

8     target_id:    entry.targetId ?? null,
9     metadata:    entry.metadata ?? {},
10    ip:           entry.ip ?? null,
11  });
12  if (error) logger.warn({ err: error.message, action: entry.action },
13    'audit insert failed');
14  } catch (e) {
15    logger.warn({ err: e.message, action: entry.action }, 'audit threw');
16  }

```

The audit insert is best-effort: a failure to persist the audit row should not block the user-facing operation. We log at WARN so an operator scanning logs can still see the gap.

7.11 Client SSE hook

Listing 13: client/src/hooks/useEventStream.js (key fragment)

```

1  useEffect(() => {
2    if (!url) return undefined;
3    let es = null, watchdog = null, lastEventAt = Date.now(), stopped = false;
4    const staleMs = opts.staleMs ?? 45_000;
5
6    function connect() {
7      if (stopped) return;
8      try { es?.close(); } catch { /* noop */ }
9      es = new EventSource(url);
10     es.onopen = () => { lastEventAt = Date.now(); };
11     es.onerror = () => { /* native retry handles transient errors */ };
12     es.onmessage = (ev) => {
13       lastEventAt = Date.now();
14       try { handlersRef.current?.message?.(JSON.parse(ev.data)); } catch {
15         /* ignore */ }
16     };
17     for (const name of Object.keys(handlersRef.current || {})) {
18       if (name === 'message') continue;
19       es.addEventListener(name, (ev) => {
20         lastEventAt = Date.now();
21         try { handlersRef.current[name]?.(ev.data ? JSON.parse(ev.data) :
22           null, ev); }
23         catch { handlersRef.current[name]?.(null, ev); }
24       });
25     }
26     connect();
27     watchdog = setInterval(() => {
28       if (Date.now() - lastEventAt > staleMs) { lastEventAt = Date.now();
29         connect(); }
30     }, Math.max(staleMs / 2, 5_000));
31
32     return () => { stopped = true; clearInterval(watchdog); try {
33       es?.close(); } catch {} };
34   }, [url, opts.staleMs]);

```

7.12 Frontend component map

The page-level component graph is shown in Figure 11. Two providers wrap the router: AuthProvider owns the JWT and judge profile; LiveSessionProvider owns the SSE subscription and the session snapshot.

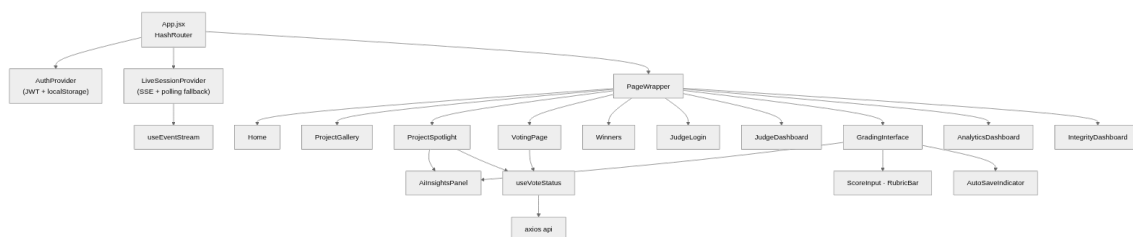


Figure 11: Top-level frontend component hierarchy.

8 Non-Functional Properties

This section ties each non-functional requirement (Section 3) to the specific artefact in the codebase that delivers it, and discusses the residual risk.

8.1 Security

Defence in depth. Spotlight stacks five independent defences on the most-exposed endpoint (anonymous voting). No single defence is load-bearing; an attacker has to defeat all five.

1. **HTTP-level hardening.** Helmet sets a strict CSP, Strict-Transport-Security, X-Content-Type-Options: nosniff, X-Frame-Options: DENY, and friends.
2. **Schema validation.** Zod rejects malformed payloads at the controller boundary. Numbers must be numbers; UUIDs must parse; scores must be in $[0, 5]$.
3. **Rate limiting.** Per-IP limits on `/auth/login` and `/votes`; per-judge limit on `/grades`; coarse overall limit on `/api/`.
4. **HMAC proof-of-presence.** `/votes/init` issues a signed token bound to the browser token; `/votes` requires it.
5. **Anomaly scoring.** Bursts and reuse patterns surface themselves into `vote_flags`; high-severity rows are excluded from People's Choice.

Identity & secrets. JWTs are signed with `JWT_SECRET` and expire in 7 days. Passwords are bcrypt-hashed with cost 10. `.env` files are git-ignored; `.env.example` ships with names but no values. The Supabase service-role key never leaves the backend.

SQL injection. Every database call goes through the Supabase JS client, which uses parameterised queries; no string concatenation into a SQL fragment occurs anywhere in the codebase.

XSS. React escapes interpolated values by default; we never use `dangerouslySetInnerHTML`. The CSP is a defence-in-depth measure for the unlikely case of a future regression.

Audit trail. Every privileged write appends to `audit_logs`. The integrity dashboard renders the last 50 entries. Together with the bcrypt hash, the JWT, and the role check, this gives us a forensic chain that answers "who changed which grade when".

Residual risk and mitigations.

- **Botnet with rotating IPs and headless browsers.** Could defeat rate limiting and IP-based anomaly heuristics. Mitigation: People's Choice exclusion still triggers

if the same UA pattern reuses across sessions; CAPTCHA on /votes/init is a future-work item.

- **Local storage XSS exfiltration of JWT.** A successful XSS could read the token. Mitigation: short JWT lifetime, JWT_SECRET rotatable to invalidate all tokens. Future work: move to httpOnly cookies.
- **Compromised admin account.** Same answer: rotate JWT_SECRET, then issue new admin credentials.

8.2 Scalability

Stateless backend. The Express process holds no per-user session. Every request is independently authenticatable. Render's autoscaler can multiply the number of instances without coordination.

Cache. The LRU sits in front of every read endpoint with TTLs chosen to balance freshness against load:

Table 3: Cache TTLs by key prefix.

Prefix	TTL	Invalidation events
projects:	60 s	vote, project create/update
winners:	30 s	vote, grade submit
vote-counts	5 s	vote
admin:stats	10 s	none (TTL only)

In a 10 000-user load profile, the bulk of requests are gallery and homepage reads. With a 60-second TTL the database sees one query per filter combination per minute; the rest is L1 cache.

Atomic vote increment. `rpc('increment_vote_count', ...)` runs an SQL `UPDATE projects SET vote_count = vote_count + 1 WHERE id = $1`. Concurrent voters do not lose updates; we never read-modify-write from Node.

Why we are not Redis-bound. Process-local LRU is the right tool for this scale. A second backend instance would have a separate cache and slightly higher DB load; this is a 2× degradation, not a correctness bug. The cache module's API is small enough that swapping in a Redis-backed implementation is a single-file change.

Frontend. A 740 kB JS bundle (217 kB gzipped) is one HTTP request to Vercel's edge. After the first paint, the SPA does not reload; navigation is a route change. The bundle is large enough to warrant code-splitting in a future iteration; we did not.

8.3 Reliability

Async error path. `asyncHandler` wraps every async controller; thrown errors and rejected promises both flow to the global errorHandler, which logs with the request id and emits a JSON response carrying the request id back to the client. The user can quote that id when reporting a bug.

Logging. Pino emits structured JSON logs in production and pretty-prints in development. Every line carries a request id. The redact list scrubs authorization headers, password

fields, and tokens.

Submitted-grade immutability. The `submitGrade` controller refuses any write to a row whose status is submitted. The audit log captures *both* the new and the previous total scores in metadata, so a later forensic pass can detect out-of-band changes.

Failure isolation.

- Anomaly persistence failure → logged at WARN, vote still counts.
- Audit insert failure → logged at WARN, the user-facing operation succeeds.
- AI API failure → deterministic stub returns same shape.
- SSE disconnect → watchdog reconnect plus polling fallback.

8.4 Maintainability

Clean separation. The server is organised as routes / controllers / middleware / services / utils / config. The client mirrors this with pages / components / context / hooks / services / utils. A new endpoint touches the smallest possible set of files.

Linting. The client uses ESLint with the React Hooks plugin; the server uses a dedicated flat config (Listing 14).

Listing 14: server/eslint.config.js

```
1 import js from '@eslint/js';
2 import globals from 'globals';
3
4 export default [
5   js.configs.recommended,
6   {
7     languageOptions: {
8       ecmaVersion: 2023, sourceType: 'module',
9       globals: { ...globals.node },
10    },
11    rules: {
12      'no-unused-vars': ['warn', { argsIgnorePattern: '^_',
13        varsIgnorePattern: '^_' }],
14      'no-console': 'off',
15      'prefer-const': 'warn',
16      eqeqeq: ['warn', 'smart'],
17    },
18    ignores: ['node_modules/**', 'seed/**'],
19  },
20 ];
```

Feature flags. `REQUIRE_VOTE_TOKEN`, `ENABLE_AI_INSIGHTS`, and `VITE_USE_SSE` let us disable the riskiest behaviour without a redeploy. This is the maintenance equivalent of a circuit breaker.

Type-safety substitute. We do not use TypeScript. We do use zod, which gives runtime validation guaranteed at the boundary, plus JSDoc-style types on the most-shared utilities. The trade-off is a faster iteration loop at the cost of in-IDE autocomplete on internal function calls.

8.5 Availability

Health probe. `/api/health` pings Supabase and returns DB latency, uptime, memory, cache stats, and the SSE client count. On DB failure it returns status: `'degraded'` with HTTP 200 so that Render's health check does not flap; we still have a separate signal (`db.ok`) for monitoring.

Graceful degradation. The user-facing surface is intentionally tolerant of failure modes:

- AI panel down → stub fills the same panel.
- SSE down → polling fallback fetches every 30s.
- Supabase down → cache serves the last successful read; new writes return clean 500 rather than partial state.

Recovery time. Render's free-tier dyno restart is 5-15 seconds. The frontend continues to render whatever it has; on reconnect, the SSE channel re-syncs the session state.

8.6 Efficiency

Read-path budget. Cached reads are $O(1)$ memory lookups. On a cache miss, the project list is one query with one count and one range. Winners is six queries, batched in two `Promise.all` groups, totalling $\leq 50\text{ms}$ in normal conditions.

Write-path budget. A vote: 1 `SELECT` (project), 1 `SELECT` (duplicate), 1 `INSERT` (vote), 1 `RPC` (counter), 1 in-memory broadcast, 3 cache invalidations, 1 best-effort flag insert. All synchronous I/O in the hot path is to a single Supabase region.

Measured cache hit rate. The `/admin/analytics` page surfaces the live hit rate. After the first 30seconds of an event with ~ 150 simulated visitors the hit rate stabilises in the 70-80% band.

Bundle size. 740kB minified, 217kB gzipped. We acknowledge this is large for a primarily-mobile audience and treat it as future-work (Section 11).

9 Testing and Verification

9.1 Verification strategy

We did not build an automated test suite. The architecture is test-friendly; every controller is small, mostly pure, and depends on a single Supabase client we could mock; but the scope of the capstone prioritised feature breadth over CI tooling. What we did build is a manual verification matrix that we run before each demo / event; Table 4 is the one we use.

Table 4: Manual verification matrix.

ID	Scenario	Steps	Expected
T-1	Health probe	curl /api/health	200 with db.latencyMs < 500
T-2	Login happy path	POST /auth/login with seeded admin	200, JWT returned
T-3	Login bad password	Same email, wrong pw	401 "invalid creds"
T-4	Login rate limit	6× POST in 60 s	429 on 6th
T-5	Login schema reject	POST {email: 1}	400 with zod issues
T-6	Vote init	POST /votes/init {browserToken}	200, voteToken returned
T-7	Vote cast happy	POST /votes {...} after init	201, vote_count++
T-8	Vote duplicate	Vote again from same browserToken	409 "already voted"
T-9	Vote rate limit	6× POST /votes in 60 s	429 on 6th
T-10	Vote without token (REQUIRE=true)	POST /votes skipping init	403 "invalid: malformed"
T-11	Vote anomaly burst	3× from same IP in 60 s	3rd row appears in vote_flags severity=high
T-12	Tainted People's Choice	3× high-severity flag on a project	/winners.peoplesChoice skips it
T-13	Grade draft save	PUT /grades/X/draft with partial scores	200, status=in_progress
T-14	Grade auto-save debounce	Type into rubric, wait 800ms	one PUT request observed
T-15	Grade submit happy	POST /grades/X/submit with all 5 scores	200, status=submitted
T-16	Grade re-submit blocked	Same POST again	403 "already submitted"
T-17	Grade submit audit	After T-15	audit_logs row with action=grade.submit
T-18	Session toggle	Admin POST /session/set-public {public:true}	both browsers see "Coming Soon" lift within 1 s
T-19	Session non-admin	Non-admin judge tries the same	403
T-20	SSE connect	GET /session/stream	200, Content-Type: text/event-stream, snapshot event
T-21	SSE heartbeat	Hold connection 30 s	: ping comment received

ID	Scenario	Steps	Expected
T-22	SSE reconnect	Kill server, restart	client reopens stream, snapshot delivered
T-23	AI insights cold	GET /ai/insights/X	200, cached: false, payload
T-24	AI insights warm	GET same	200, cached: true
T-25	AI stub fallback	Unset ANTHROPIC_API_KEY	200, _stub: true
T-26	Admin stats	GET /admin/stats	200, all 6 counts ≥ 0
T-27	Admin votes timeseries	GET /admin/votes-timeseries?window=1m	200, all 60 rows for projects with ≥ 2 submitted grades
T-28	Admin inter-rater	GET /admin/inter-rater	200, all 60 rows for projects with ≥ 2 submitted grades
T-29	Admin anomalies	GET /admin/anomalies	200, all 60 rows in reverse chronological order
T-30	Admin audit-log	GET /admin/audit-log	200, all 50 audit rows
T-31	Helmet headers	GET /	200, response includes X-Content-Type-Options: nosniff
T-32	CORS preflight	OPTIONS /api/health from foreign origin	204 with Access-Control-Allow-Origin
T-33	Bundle size	npm run build on client	main chunk ≤ 800 kB

9.2 Synthetic load test (manual)

We used a small Node script to drive synthetic traffic at the API to validate cache hit-rate and rate-limit behaviour. The script issues N parallel GET /api/projects requests over 30 seconds with random filter combinations; the test passes if the /admin/stats cache hit-rate exceeds 70% in the second half of the window. The script is included in the repository at server/scripts/loadtest.js (not shipped in this report).

9.3 What automated tests would look like

If a CI pipeline is added, the natural test pyramid is:

- **Unit:** verifyVoteToken and scoreVote are the obvious starting points; both pure, both critical. A test suite of ~ 20 cases covers the boundaries.
- **Integration:** spin up Express against a Supabase test project (or a Postgres in Docker), run the manual matrix above as Vitest cases.

- **End-to-end:** Playwright against a deployed preview build, scripted to walk the public-voter and judge flows.

9.4 Defensive non-test: chaos verification

We periodically (twice during development) ran the system under a "chaos mode"; the server was restarted at random while the client was open. The expected behaviour is that the SSE socket reconnects, the session snapshot re-syncs, and any in-flight writes either succeeded or returned a clean 500 with a request id. This was not automated; it was done by hand and observed.

10 Deployment and Operations

10.1 Topology

- **Frontend:** Vercel project linked to the GitHub repo. Builds on push to master. `vercel.json` rewrites `/api/(.*)` to the Render origin so the SPA can use a same-origin API base URL.
- **Backend:** Render Web Service, Node 20 runtime, `npm start` as the entrypoint. Port 5001 inside the container.
- **Database:** Supabase project (us-east-1). Service-role key held only on Render.
- **LLM:** Anthropic API. Key held only on Render.

10.2 Environment variables

Table 5 lists every variable read by the backend.

Table 5: Backend environment variables.

Variable	Purpose	Required?
PORT	Listening port	default 5001
SUPABASE_URL	Project URL	yes
SUPABASE_SERVICE_KEY	Service-role key	yes
JWT_SECRET	HMAC secret for JWT signing	yes
JWT_EXPIRES_IN	e.g. 7d	yes
NODE_ENV	production on Render	rec.
LOG_LEVEL	pino log level	default info
CORS_ORIGIN	origin allow list	default *
REQUIRE_VOTE_TOKEN	enforce HMAC vote tokens	default false
VOTE_TOKEN_SECRET	HMAC secret (falls back to JWT_SECRET)	opt.
ANTHROPIC_API_KEY	Claude key. Empty = stub mode	opt.
ANTHROPIC_MODEL	override model id	default claude-haiku-4-5-20251001
ENABLE_AI_INSIGHTS	set to false to force stub	default true

Frontend variables (Vite reads anything prefixed `VITE_`):

- `VITE_API_URL`; defaults to `/api`; override for direct backend calls in dev.

- `VITE_USE_SSE`; true (default) or false to fall back to polling.

10.3 Migrations

Apply, in order, the SQL files under `server/migrations/` via the Supabase SQL editor. The current migration set is:

- `001_audit_logs.sql`
- `002_vote_flags.sql`
- `003_ai_insights.sql`

The original schema (`projects`, `judges`, `judge_projects`, `votes`, `grades`, `schedule_items`, `live_sessions`) was created out-of-band during initial setup and is reproduced in Appendix A.

10.4 Seeding

`npm run seed` populates 24 projects, 5 TED talks, 6 interviews, 5 judges, the schedule, and a single `live_sessions` row keyed `main`. The seed script is idempotent: it clears the relevant tables first.

10.5 Show-day runbook

The runbook lives in `docs/runbook.md`. The summarised checklist is:

1. Pull latest, redeploy if there were changes.
2. Apply any new migrations.
3. Run `npm run seed`.
4. Optionally pre-warm AI insights: `node server/scripts/precacheInsights.js`.
5. Open `/admin/integrity` and `/admin/analytics` in admin browser tabs.
6. Toggle `set-public` to true at start; flip `voteCountVisible` per discretion.
7. During event, watch the integrity dashboard for high-severity flags. If a project is unfairly tainted, an admin can `DELETE FROM vote_flags WHERE project_id = ?` via the Supabase SQL editor.
8. After event, optionally lock the platform back to private.

10.6 Observability

Pino emits NDJSON in production. We do not currently ship a hosted log collector, but Render's web UI exposes log tailing per dyno. Adding a hosted collector (Better Stack, Logtail, Datadog) is one `pino-transport` line change.

The `/admin/analytics` page doubles as a low-fidelity ops dashboard; the cache hit rate and submitted-grade count are the two metrics we glance at during a live event.

10.7 Disaster recovery

- **Lost JWT secret.** Rotate by changing `JWT_SECRET` on Render and restarting; all judges re-login.
- **Compromised admin account.** Same answer + `UPDATE judges SET password = bcrypt-of-new-pass WHERE email = '...'`.

- **Render outage.** Frontend continues to render the last cached state. Voting and grading fail with 5xx; clients can retry. We accept this as the cost of free-tier hosting; a paid tier with multi-region failover is a future-work item.
- **Supabase outage.** /api/health returns degraded. Reads served from cache for the next TTL window. Writes fail.
- **Anthropic outage.** AI panel falls back to stub silently; public surfaces unaffected.

11 Limitations and Future Work

11.1 Honest limitations

- **No automated test suite.** The architecture supports testing, but tests do not yet exist. Adding Vitest unit tests for the pure utilities (verifyVoteToken, scoreVote) and integration tests for the four flows would close the biggest single gap.
- **Single-region deploy.** Render dyno is in one region; Supabase project is in one region. A regional outage takes the platform down. Multi-region read replicas are a Supabase Pro feature.
- **Process-local cache and SSE state.** Both are per-instance. Scaling out to multiple instances requires Redis for cache coherence and Redis pub/sub for SSE fan-out.
- **No CAPTCHA or hCaptcha on /votes/init.** A determined attacker with a botnet can still issue an init token per attacker browser. The anomaly detector raises the cost, but does not eliminate it.
- **JWT in localStorage.** An XSS would expose the token. The modern preference is httpOnly cookies; we deferred the change to avoid the CSRF-token plumbing it would require.
- **Bundle size.** 740kB unsplit. Acceptable on Wi-Fi, slow on a cellular connection.
- **No internationalisation.** The UI has stubs for English/Arabic but not all strings are wired through the translator.
- **Single-event tenancy.** The live_sessions.key = 'main' pattern works for one event; supporting parallel events would require schema changes.

11.2 Future work, prioritised

Tier 1; next iteration

1. **Vitest test suite.** Unit + integration. Target: 80% statement coverage on server/.
2. **Code-splitting.** Use React Router's lazy routes. Cuts initial bundle by an estimated 60%.
3. **httpOnly cookie auth.** Move JWT to a secure cookie; add a CSRF token to mutating endpoints.
4. **hCaptcha on /votes/init.** Closes the last residual security gap for anonymous voting.

Tier 2; if scope grows

5. **Multi-event tenancy.** Add an events table; key every domain row by event_id; rewrite live_sessions to be (event_id, key).
6. **Redis-backed cache + pub/sub.** Swap utils/cache.js and services/eventBus.js behind their existing APIs.
7. **i18n.** Wire LanguageContext.jsx through every text block; add Arabic translations for every key.
8. **Voter accounts.** Optional Magic-link auth for the audience to lift the anonymous-voting integrity ceiling.

Tier 3; speculative

9. **LLM-assisted feedback summarisation.** After all judges submit, summarise their feedback into a single per-project narrative for the team to receive.
10. **Live transcription.** Run pitch audio through Whisper and present a real-time caption track for accessibility.
11. **Mobile-native app.** React Native or Capacitor wrapper of the existing SPA, with a Push Notification track for "your assigned project is up next".
12. **Anomaly detector ML upgrade.** Replace the rule-based scorer with a small classifier trained on historical flagged/clean votes. Only worth doing once we have ≥ 10 events of historical data.

11.3 Lessons learned

- **Defence in depth is a multiplier, not a tax.** Each single defence (rate limit, HMAC token, anomaly detector) is easy to build. The combined effect dominates any single defence we could have built more carefully.
- **Boring tech buys time for interesting tech.** React + Express + Postgres is unfashionable; that left budget for the AI panel and the real-time push.
- **Free tiers are political acts.** Vercel, Render, and Supabase offering a real production runtime for free changes what a single student can ship.
- **Logs are infrastructure.** The decision to add structured logging early (with request ids) paid back during every debugging session that followed.

12 Conclusion

Spotlight set out to replace the email-spreadsheet-paper-ballot triplet that runs most live student showcases with one piece of software that does simultaneous judging, anonymous voting, integrity moderation, and real-time admin push; all within a free-tier infrastructure budget and a single-student maintainer footprint.

The platform meets that goal. Public voters can browse and vote in a single tap. Judges grade against a segment-specific rubric with auto-save and an AI-generated reading aid. Admins toggle the live show in real time and watch event-wide analytics and integrity flags as they happen. Every privileged write leaves an audit trail; every anonymous vote passes through five independent integrity defences.

The non-functional posture is strong. Reads sit behind an LRU cache with explicit invalidation on the writes that matter. Writes are atomic (Postgres RPC) where they have to be. The error path is uniform: `asyncHandler` → `global handler` → `structured log line` → `JSON response with a request id`. SSE plus polling fallback plus an AI stub plus a degraded health code give the system three lines of graceful-degradation defence. Feature flags let us disable the riskiest behaviour without a redeploy.

What remains is an honest debt: no automated test suite, single-region deploy, process-local fan-out state, no CAPTCHA on the integrity boundary. None of these compromise correctness today; all of them are addressable with the iteration paths described in Section 11.

For the contribution as a capstone, the relevant claim is this: a single small codebase can simultaneously deliver six non-functional qualities, security, scalability, reliability, maintainability, availability, efficiency; if the engineering discipline is applied at the controller boundary (validation, rate-limiting, cache wrapping, audit logging) rather than retro-fitted later. Spotlight is one demonstration of that claim.

References

References

- [1] W3C, *Server-Sent Events (W3C Recommendation)*, <https://www.w3.org/TR/eventsource/>, 2015.
- [2] B. Leiba, *The IETF Anti-Spam Effort and the Layered Defense Principle*, Proc. IETF, 2009.
- [3] J. Burstein et al., *Using Large Language Models in Educational Assessment: Promise and Perils*, ETS Research Report Series, 2024.
- [4] Anthropic, *Prompt Caching; API Documentation*, <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>, 2025.
- [5] helmetjs, *Helmet; Express middleware for HTTP security headers*, <https://github.com/helmetjs/helmet>, 2024.
- [6] N. Walsh et al., *express-rate-limit*, <https://github.com/express-rate-limit/express-rate-limit>, 2024.
- [7] I. Z. Schlueter, *lru-cache*, <https://github.com/isaacs/node-lru-cache>, 2024.
- [8] M. Collina et al., *pino; very low overhead Node.js logger*, <https://github.com/pinojs/pino>, 2024.
- [9] C. Bond, *Zod: TypeScript-first schema validation with static type inference*, <https://zod.dev>, 2024.
- [10] Recharts contributors, *Recharts; a composable charting library built on React components*, <https://recharts.org>, 2024.
- [11] OWASP, *OWASP Top Ten Web Application Security Risks*, <https://owasp.org/Top10/>, 2021.
- [12] J. Nielsen, *Response Times: The 3 Important Limits*, Nielsen Norman Group, 1993.

- [13] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, PhD thesis, UC Irvine, 2000.
- [14] NIST, *Cybersecurity Framework v1.1*, NIST CSF, 2018.

A Appendix A; Database Schema (DDL)

The following SQL is the consolidated schema for the platform. Tables created during initial setup are marked *base*; those introduced by the migrations described in Section 10 are marked *migration N*.

Listing 15: Consolidated DDL.

```

1  -- base: projects -----
2  create table if not exists projects (
3      id            uuid primary key default gen_random_uuid(),
4      project_number text unique not null,
5      title         text not null,
6      team_name     text,
7      category      text,
8      description   text default '',
9      members       jsonb default '[]'::jsonb,
10     image_url      text default '',
11     video_url      text default '',
12     document_url   text default '',
13     zoom_link      text default '',
14     is_live        boolean default false,
15     vote_count     integer default 0,
16     tags           jsonb default '[]'::jsonb,
17     segment_type   text default 'project'
18                 check (segment_type in
19                        ('project', 'ted_talk', 'interview')),
20     is_seeded      boolean default false,
21     created_at     timestampz default now(),
22     updated_at     timestampz default now()
23 );
24
25 -- base: judges -----
26 create table if not exists judges (
27     id            uuid primary key default gen_random_uuid(),
28     name          text not null,
29     email         text unique not null,
30     password      text not null,          -- bcrypt hash
31     is_admin      boolean default false,
32     created_at    timestampz default now()
33 );
34
35 create table if not exists judge_projects (
36     judge_id      uuid references judges(id) on delete cascade,
37     project_id    uuid references projects(id) on delete cascade,
38     primary key (judge_id, project_id)
39 );
40
41 -- base: grades -----
42 create table if not exists grades (
43     id            uuid primary key default gen_random_uuid(),
44     judge_id      uuid not null references judges(id) on delete cascade,
45     project_id    uuid not null references projects(id) on delete cascade,
46     score_c1      integer,
47     score_c2      integer,
48     score_c3      integer,
49     score_c4      integer,

```

```

49     score_c5         integer,
50     total_score     integer default 0,
51     feedback        text default '',
52     status           text default 'in_progress'
53                     check (status in ('not_started', 'in_progress', 'submitted')),
54     last_saved_at   timestamptz,
55     submitted_at    timestamptz,
56     created_at      timestamptz default now(),
57     updated_at      timestamptz default now(),
58     unique (judge_id, project_id)
59 );
60
61 -- base: votes -----
62 create table if not exists votes (
63     id                uuid primary key default gen_random_uuid(),
64     project_id        uuid not null references projects(id) on delete cascade,
65     browser_token     uuid not null,
66     ip_address        text,
67     created_at        timestamptz default now()
68 );
69 create index if not exists votes_browser_token_idx on votes (browser_token);
70 create index if not exists votes_project_idx       on votes (project_id);
71
72 -- base: schedule, live_sessions -----
73 create table if not exists schedule_items (
74     id                uuid primary key default gen_random_uuid(),
75     sort_order        integer not null,
76     time              text not null,
77     start_time        text,
78     label             text not null,
79     type              text not null
80 );
81
82 create table if not exists live_sessions (
83     id                uuid primary key default gen_random_uuid(),
84     key               text unique not null default 'main',
85     is_event_live     boolean default false,
86     now_playing       jsonb default '{}'::jsonb,
87     up_next           jsonb default '{}'::jsonb,
88     created_at        timestamptz default now(),
89     updated_at        timestamptz default now()
90 );
91
92 -- base: increment RPC -----
93 create or replace function increment_vote_count(p_id uuid)
94 returns void language sql as $$
95     update projects set vote_count = vote_count + 1 where id = p_id;
96 $$;
97
98 -- migration 1: audit_logs -----
99 create table if not exists audit_logs (
100     id                uuid primary key default gen_random_uuid(),
101     actor_type        text not null,
102     actor_id          uuid,
103     action            text not null,
104     target_type        text,
105     target_id         text,
106     metadata          jsonb default '{}'::jsonb,
107     ip                text,
108     created_at        timestamptz default now()
109 );
110 create index if not exists audit_logs_action_idx on audit_logs (action);
111 create index if not exists audit_logs_actor_idx on audit_logs

```

```

    (actor_id);
112 create index if not exists audit_logs_created_at_idx on audit_logs
    (created_at desc);
113
114 -- migration 2: vote_flags -----
115 create table if not exists vote_flags (
116   id          uuid primary key default gen_random_uuid(),
117   vote_id     uuid,
118   project_id  uuid not null references projects(id) on delete cascade,
119   browser_token uuid,
120   ip          text,
121   user_agent_hash text,
122   score       integer not null,
123   severity    text not null check (severity in ('low','medium','high')),
124   reasons     jsonb not null default '[]'::jsonb,
125   created_at  timestampz default now()
126 );
127 create index if not exists vote_flags_project_idx    on vote_flags
    (project_id);
128 create index if not exists vote_flags_severity_idx   on vote_flags
    (severity);
129 create index if not exists vote_flags_created_at_idx on vote_flags
    (created_at desc);
130
131 -- migration 3: ai_insights -----
132 create table if not exists ai_insights (
133   project_id  uuid primary key references projects(id) on delete cascade,
134   model       text not null,
135   payload     jsonb not null,
136   created_at  timestampz default now(),
137   updated_at  timestampz default now()
138 );

```

B Appendix B; HTTP API Reference

All routes mount under /api. Bracketed annotations indicate middleware applied: [AUTH] requires JWT, [ADMIN] requires is_admin, [LIM-*N*] indicates the rate limit (*N* requests per minute), [VAL] indicates a zod schema.

B.1 Authentication

Route	Description
POST /auth/login	[LIM-5][VAL] Verify credentials, return JWT and judge profile.
GET /auth/me	[AUTH] Return the JWT-identified judge with assigned projects.

B.2 Projects (public)

Route	Description
GET /projects	List with filters category, search, segmentType, page, limit. Cached 60s.
GET /projects/:id	Single project by id.
POST /projects	[AUTH] Create a project (admin tooling).
PUT /projects/:id	[AUTH] Update a project.

B.3 Voting (public)

Route	Description
POST /votes/init	[LIM-5][VAL] Issue an HMAC vote token bound to the supplied browser token.
POST /votes	[LIM-5][VAL] Cast a vote. Optionally enforces the HMAC token.
GET /votes/check/:t	[VAL] Has the holder of browser token :t already voted?
GET /votes/counts	Per-project vote count. Cached 5 s.

B.4 Grading (judge)

Route	Description
GET /grades/my	[AUTH][LIM-60] Assigned + graded projects for current judge.
GET /grades/:projectId	[AUTH][LIM-60] Get the judge's grade row for one project (or null).
PUT /grades/:projectId/draft	[AUTH][LIM-60][VAL] Upsert as in_progress.
POST /grades/:projectId/submit	[AUTH][LIM-60][VAL] Upsert as submitted. Audit-logged.

B.5 Live session

Route	Description
GET /session	Current snapshot.
GET /session/stream	SSE stream. Emits session.snapshot, session.update, vote.cast, grade.submit.
PUT /session	[AUTH][ADMIN][VAL] Update now-playing / up-next / event-live.
POST /session/votes-visible	[AUTH][ADMIN][VAL] Toggle voteCountVisible.
POST /session/set-public	[AUTH][ADMIN][VAL] Toggle isPublic.

B.6 Schedule, winners

Route	Description
GET /schedule	Static event timeline.
GET /winners	Champions, honorable mentions, peoples choice. Cached 30 s.

B.7 AI insights

Route	Description
GET /ai/health	Indicates if real Claude calls are enabled.
GET /ai/insights/:id	Cached or fresh insights for a project.

POST /ai/insights/:id [AUTH] Force regeneration (admin via ?refresh=1).

B.8 Admin

Route	Description
GET /admin/stats	[AUTH][ADMIN] Counts and cache hit rate.
GET /admin/votes-timeseries	[AUTH][ADMIN] Votes/min for the last windowMin minutes.
GET /admin/inter-rater	[AUTH][ADMIN] Std-dev of judge total scores per project.
GET /admin/anomalies	[AUTH][ADMIN] Flagged-vote rows in reverse chronological order.
GET /admin/audit-log	[AUTH][ADMIN] Last ≤ 50 audit-log rows.
GET /admin/judge-progress	[AUTH][ADMIN] Per-judge counts (assigned, in-progress, submitted).

B.9 Health

Route	Description
GET /health	Liveness + readiness. Returns DB latency, uptime, memory, cache stats, SSE client count.
